

Bachelor-Thesis

zur Erlangung des akademischen Grades

Bachelor of Science (B. Sc.)

an der Hochschule für Technik und Wirtschaft des Saarlandes

im Studiengang Praktische Informatik
der Fakultät für Ingenieurwissenschaften

Netzwerk-Monitoring Dashboard mit Automatisierung

vorgelegt von

Xudong Zhang

betreut und begutachtet von

Prof. Dr.-Ing. André Miede

Andreas Schaffhauser

Saarbrücken, den 16. September 2026

Selbständigkeitserklärung

Ich versichere, dass ich die vorliegende Arbeit (bei einer Gruppenarbeit: den entsprechend gekennzeichneten Anteil der Arbeit) selbständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt habe.

Ich erkläre hiermit weiterhin, dass die vorgelegte Arbeit zuvor weder von mir noch von einer anderen Person an dieser oder einer anderen Hochschule eingereicht wurde.

Darüber hinaus ist mir bekannt, dass die Unrichtigkeit dieser Erklärung eine Benotung der Arbeit mit der Note „nicht ausreichend“ zur Folge hat und einen Ausschluss von der Erbringung weiterer Prüfungsleistungen zur Folge haben kann.

Saarbrücken, den 16. September 2026

Xudong Zhang

Zusammenfassung

Kurze Zusammenfassung des Inhaltes in deutscher Sprache, der Umfang beträgt zwischen einer halben und einer ganzen DIN A4-Seite.

Orientieren Sie sich bei der Aufteilung bzw. dem Inhalt Ihrer Zusammenfassung an Kent Becks Artikel: <http://plg.uwaterloo.ca/~migod/research/beck00PSLA.html>.

*We have seen that computer programming is an art,
because it applies accumulated knowledge to the world,
because it requires skill and ingenuity, and especially
because it produces objects of beauty.*

— Donald E. Knuth [16]

Danksagung

Hier können Sie Personen danken, die zum Erfolg der Arbeit beigetragen haben, beispielsweise Ihren Betreuern in der Firma, Ihren Professoren/Dozenten an der htw saar, Freunden, Familie usw.

Inhaltsverzeichnis

1	Einleitung	1
1.1	Motivation und Zielsetzung	1
1.2	Problemstellung	2
1.3	Ziele der Arbeit	2
1.3.1	Pflichtziele	3
1.3.2	Optionale Erweiterungen	4
1.4	Geplantes Vorgehen	4
1.5	Aufbau der Arbeit	5
2	Grundlage	7
2.1	Grundbegriffe und Definitionen	7
2.1.1	Netzwerk-Monitoring, Verfügbarkeit und Latenz	7
2.1.2	Metrikbasiertes Monitoring und Alerting	7
2.1.3	Containerisierung und TCP-basierte Erreichbarkeitsprüfung	8
2.2	Technologische Grundlagen	8
2.2.1	Spring Boot, Actuator und Micrometer	8
2.2.2	Prometheus, Grafana und Alertmanager	9
2.2.3	Containerisierte Bereitstellung mit Docker und Docker Compose	9
2.3	Verwendete Methoden und Konzepte des entwickelten Systems	9
2.3.1	Konfigurationsgetriebenes Gerätemanagement und zeitgesteuertes Polling	10
2.3.2	Zustandsbasierte Metriken und Kennzahlenmodell	10
2.3.3	Schichtenarchitektur, Fehlerbehandlung und Robustheit	10
2.3.4	Automatisierte Tests der Kernlogik	11
2.4	Stand der Technik und Einordnung	11
3	Analyse	13
3.1	Analyse des Problemraums	13
3.2	Anforderungsermittlung	13
3.2.1	Funktionale Anforderungen	14
3.2.2	Nichtfunktionale Anforderungen	14
3.3	Stand der Technik und verwandte Arbeiten	15
3.3.1	Prometheus	15
3.3.2	Prometheus Blackbox Exporter	15
3.3.3	Grafana	15
3.3.4	Alertmanager	15
3.3.5	Kommerzielle und Open-Source-Monitoring-Plattformen	16
3.3.6	Forschungsarbeiten	16
3.3.7	Abdeckungsanalyse und identifizierte Lücken	16
3.4	Zusammenfassung	17
4	Konzeption	19
4.1	Überblick über die Gesamtarchitektur	19

4.2	Schließen der identifizierten Lücken	20
4.2.1	Lücke 1: Fehlende Incident-Historie	20
4.2.2	Lücke 2: Fehlende integrierte REST-Schnittstelle	21
4.2.3	Lücke 3: Komplexe Bereitstellung	21
4.2.4	Lücke 4: Überdimensionierung	22
4.2.5	Lücke 5: Fehlende manuelle Auslösung	22
4.3	Konzeption des Monitoring-Dienstes	22
4.3.1	Verwaltung der Überwachungsziele	22
4.3.2	Ausführung der Erreichbarkeitsprüfungen	23
4.3.3	Fehlerbehandlung und Robustheit	24
4.4	Konzeption des Metrik-Exports	24
4.4.1	Metrikmodell	24
4.4.2	Exportformat	24
4.5	Konzeption der Zeitreihenspeicherung	25
4.5.1	Speichermodell	25
4.5.2	Retentionspolitik	25
4.6	Konzeption der Visualisierung	25
4.6.1	Dashboard-Konzeption	26
4.6.2	Alert-Visualisierung	26
4.7	Konzeption der Alert-Komponente	26
4.7.1	Alert-Regeln	26
4.7.2	Alert-Weiterleitung	27
4.8	Konzeption des Incident-Speichers	27
4.8.1	Incident-Modell	27
4.8.2	Incident-Lifecycle	27
4.9	Konzeption der REST-Schnittstelle	28
4.9.1	Endpunkte	28
4.9.2	Sicherheit	29
4.10	Konzeption der containerisierten Bereitstellung	29
4.10.1	Komponenten und Services	29
4.10.2	Netzwerk und Volumes	30
4.11	Zusammenfassung	30
5	Implementierung	31
5.1	Umsetzungstechnologien	31
5.1.1	Java und Spring Boot	31
5.1.2	Maven	32
5.1.3	Persistenz	32
5.1.4	Docker und Docker Compose	32
5.2	Anwendungsstruktur	33
5.3	Konfiguration der Überwachungsziele	33
5.4	Implementierung der Geräteprüfung	34
5.4.1	Prüfung der Erreichbarkeit	34
5.4.2	Fehlerbehandlung	35
5.5	Zeitgesteuerte Ausführung	36
5.6	Modellierung und Persistierung	36
5.6.1	Gerätemodell	36
5.6.2	Prüfergebnis	37
5.7	Implementierung der REST-Schnittstellen	37
5.7.1	Geräte- und Monitoring-Schnittstellen	37
5.7.2	Manuelle Prüfungen	38

5.7.3	Sicherheit der API	38
5.8	Metriken und Monitoring-Integration	38
5.9	Alert-Webhooks und Incident-Management	39
5.9.1	Verarbeitung eingehender Alerts	39
5.9.2	Incident-Modell	39
5.9.3	REST-Abfrage der Incidents	40
5.10	Globale Fehlerbehandlung	40
5.11	Containerisierte Bereitstellung	40
5.11.1	Persistente Daten	41
5.12	Proof-of-Concept	41
5.13	Zusammenfassung	42
6	Evaluation	43
7	Zusammenfassung und Ausblick	45
	Literatur	47
	Abbildungsverzeichnis	49
	Tabellenverzeichnis	49
	Listings	49
	Abkürzungsverzeichnis	51
A	Erster Abschnitt des Anhangs	55

1 Einleitung

1.1 Motivation und Zielsetzung

Die fortschreitende Digitalisierung betrieblicher Abläufe führt dazu, dass die Verfügbarkeit von Netzwerkdiensten und Anwendungen in modernen IT-Umgebungen eine zentrale Voraussetzung für einen stabilen Arbeitsbetrieb darstellt. Bereits kleinere Störungen in Netzwerkverbindungen oder auf Anwendungsebene können dazu führen, dass Arbeitsprozesse unterbrochen, Reaktionszeiten verlängert oder zentrale IT-Dienste nur eingeschränkt nutzbar werden. Vor diesem Hintergrund gewinnt ein systematisches und möglichst automatisiertes Monitoring zunehmend an Bedeutung.

Die Wahl des Themas „Netzwerk-Monitoring Dashboard mit Automatisierung“ ergibt sich aus mehreren fachlichen und praktischen Aspekten.

- **Praxisbezug aus dem Praktikum:** Im Rahmen des IT-Service-Praktikums beim Bildungscampus Saarland wurde deutlich, wie stark Netzwerkstörungen den laufenden Betrieb beeinträchtigen können. Dabei zeigte sich zugleich die Relevanz eines modernen und automatisierten Netzwerk-Monitorings.
- **Berufliche Relevanz:** Die angestrebte fachliche Ausrichtung liegt im Bereich DevOps und Netzwerkinfrastruktur. Monitoring stellt in diesem Umfeld eine zentrale Kompetenz dar und besitzt zugleich eine hohe Bedeutung auf dem aktuellen IT-Arbeitsmarkt.
- **Technisches Interesse:** Das Thema verbindet zentrale Interessensfelder der praktischen Informatik, insbesondere Netzwerkinfrastruktur, Automatisierung sowie moderne Werkzeuge des IT-Betriebs.

Der Praxisbezug des Themas ist besonders relevant, da Monitoring im IT-Service nicht nur der technischen Beobachtung von Systemen dient, sondern unmittelbar zur Sicherstellung betrieblicher Abläufe beiträgt. Werden Netzwerkprobleme oder Leistungsabfälle zu spät erkannt, können Helpdesk-Prozesse, Fernzugriffe, interne Verwaltungsanwendungen oder weitere digitale Dienste verzögert oder gestört werden. Ein automatisiertes Monitoring kann in solchen Situationen dazu beitragen, Störungen früher zu erkennen, Zustände transparenter darzustellen und die Reaktionsfähigkeit des IT-Betriebs zu verbessern.

Darüber hinaus besitzt das Thema auch aus fachlicher Sicht eine hohe Relevanz, da es mehrere Kernbereiche der praktischen Informatik miteinander verbindet. Dazu zählen unter anderem verteilte Systeme, Netzwerkinfrastruktur, automatisierte Datenerfassung, Zeitreihenverarbeitung, Visualisierung und einfache Alarmierungsmechanismen. Die Arbeit greift damit sowohl technische als auch organisatorische Fragestellungen auf und eignet sich daher besonders für eine praxisnahe Abschlussarbeit mit wissenschaftlichem Anspruch.

Das übergeordnete Ziel der Arbeit besteht darin, ein nachvollziehbares und technisch realisierbares Konzept für ein Monitoring-System zu entwickeln und dessen Umsetzbarkeit anhand eines Prototyps zu demonstrieren. Neben der rein technischen Realisierung soll auch gezeigt werden, wie sich verschiedene Werkzeuge sinnvoll kombinieren lassen,

1 Einleitung

um einen konsistenten Monitoring-Prozess von der Datenerfassung bis zur Visualisierung und Alarmierung abzubilden.

1.2 Problemstellung

In modernen IT-Infrastrukturen ist die kontinuierliche Überwachung von Netzwerkverbindungen, insbesondere im LAN- und VPN-Bereich, sowie des Zustands von Anwendungen von zentraler Bedeutung für einen stabilen und zuverlässigen Betrieb. Gerade in IT-Service-Umgebungen können Netzwerkstörungen oder Fehler auf Anwendungsebene in kurzer Zeit zu erheblichen betrieblichen Beeinträchtigungen führen. Dies gilt insbesondere dann, wenn mehrere Systeme voneinander abhängen und Störungen nicht isoliert, sondern kettenartig auf andere Dienste wirken.

In vielen IT-Service-Abteilungen werden Netzwerkprobleme jedoch weiterhin vor allem durch manuelle Ping-Tests oder unregelmäßige Einzelprüfungen erkannt. Solche Verfahren können zwar in einfachen Situationen kurzfristig hilfreich sein, sie sind jedoch nur bedingt geeignet, um größere oder dynamische Infrastrukturen zuverlässig zu überwachen. Die fehlende Automatisierung hat zur Folge, dass Störungen verspätet entdeckt werden, der manuelle Prüfaufwand hoch bleibt, historische Daten für weiterführende Analysen fehlen und bei Ausfällen keine unmittelbare Alarmierung erfolgt.

Diese Vorgehensweise ist nicht nur ineffizient, sondern auch fehleranfällig. Manuelle Kontrollen sind stark vom Zeitpunkt der Durchführung, vom Erfahrungswissen der Beteiligten und von der systematischen Dokumentation einzelner Beobachtungen abhängig. Dadurch entsteht das Risiko, dass kurzzeitige Ausfälle unbemerkt bleiben oder dass Zusammenhänge zwischen verschiedenen Fehlerbildern im Nachhinein nicht mehr zuverlässig rekonstruiert werden können.

Hinzu kommt, dass ohne kontinuierlich gespeicherte Messwerte nur eingeschränkt beurteilt werden kann, ob ein Problem einmalig aufgetreten ist oder ob sich bereits über längere Zeit ein negativer Trend abgezeichnet hat. Für einen professionellen IT-Betrieb ist jedoch nicht nur die unmittelbare Störungserkennung relevant, sondern auch die Möglichkeit, Entwicklungen über einen längeren Zeitraum nachzuvollziehen, Grenzwerte zu definieren und daraus gezielte Maßnahmen abzuleiten.

Ein professioneller IT-Betrieb erfordert daher ein Monitoring, das automatisiert, kontinuierlich und nachvollziehbar arbeitet. Es sollte Metriken auf unterschiedlichen Ebenen erfassen, diese zentral speichern, für operative und analytische Zwecke visualisieren und bei kritischen Zuständen eine zeitnahe Alarmierung ermöglichen. Genau an dieser Stelle setzt die vorliegende Arbeit an, indem ein prototypisches Monitoring-System für Netzwerk- und Anwendungsebene konzipiert und umgesetzt wird.

1.3 Ziele der Arbeit

Ziel der Arbeit ist die Entwicklung eines Monitoringsystems für LAN-/VPN-Verbindungen sowie für eine Spring-Boot-Anwendung. Das System soll Messdaten automatisiert erfassen, speichern, visualisieren und bei kritischen Zuständen eine Alarmierung auslösen. Damit verfolgt die Arbeit das Ziel, einen durchgängigen technischen Ablauf von der Generierung über die Erfassung und Speicherung bis hin zur Auswertung und Benachrichtigung abzubilden.

Die wesentlichen Zielsetzungen lauten wie folgt:

- Erfassung von Anwendungsmetriken durch eine Spring-Boot-Komponente mit Micrometer, insbesondere HTTP-Antwortzeiten, Fehlerquoten, Thread-Anzahl, JVM-

Speicher und CPU-Auslastung.

- Überwachung zusätzlicher Netzwerkmetriken wie Ping-Latenz, Erreichbarkeit, VPN-Status und Port-Verfügbarkeit ausgewählter Dienste.
- Regelmäßige Speicherung aller Metriken in Prometheus als Zeitreihendatenbasis zur Auswertung aktueller und historischer Werte.
- Visualisierung zentraler Kennzahlen aus Anwendung, Netzwerk und Host-Systemen in einem Grafana-Dashboard.
- Umsetzung einer einfachen Alarmierungsfunktion, die bei Ausfällen oder Schwellenwertüberschreitungen automatisiert eine E-Mail-Benachrichtigung versendet.

Die einzelnen Zielsetzungen greifen ineinander. Die Datenerfassung bildet die Grundlage für jede weitere Verarbeitung, da ohne verlässliche Metriken weder eine Zeitreihenspeicherung noch eine aussagekräftige Visualisierung möglich ist. Ebenso setzt eine funktionierende Alarmierung voraus, dass Zustandsänderungen und Grenzwertverletzungen zunächst systematisch erkannt und ausgewertet werden.

Aus fachlicher Sicht verfolgt die Arbeit damit nicht nur die Implementierung einzelner Werkzeuge, sondern die Entwicklung einer durchgängigen Monitoring-Kette. Entscheidend ist dabei, dass die verschiedenen Komponenten nicht isoliert betrachtet, sondern als zusammenhängendes System verstanden werden. Dadurch wird die Arbeit über eine reine Tool-Installation hinaus erweitert und erhält einen eigenständigen konzeptionellen Mehrwert.

Um den Arbeitsumfang innerhalb des vorgegebenen Bearbeitungszeitraums realistisch zu gestalten, wird zwischen Pflichtzielen und optionalen Erweiterungen unterschieden. Diese Trennung ist notwendig, um einen klaren Mindestumfang festzulegen und zugleich weiterführende Entwicklungsmöglichkeiten transparent zu machen.

1.3.1 Pflichtziele

Die Pflichtziele definieren den Kern der Arbeit und müssen im Rahmen des Projekts umgesetzt werden. Sie bilden zugleich die Grundlage für die spätere Bewertung des Prototyps.

- **Datenerfassung:** Entwicklung einer Spring-Boot-API mit Micrometer-Anbindung zur Erfassung von JVM-Metriken wie Speicher, CPU-Auslastung und Threads sowie von HTTP-Metriken wie Antwortzeit und Fehlerquote.
- **Speicherung:** Aufbau einer Prometheus-basierten Zeitreihenspeicherung mit definierten Scraping-Intervallen, beispielsweise 15 Sekunden, um aktuelle und historische Daten auswertbar zu machen.
- **Visualisierung:** Erstellung eines Grafana-Dashboards mit Panels für Ping-Latenz, VPN-Uptime, Port-Status und JVM-Leistung.
- **Alarmierung:** Implementierung eines Bash-Skripts, das bei Schwellenwertüberschreitungen, beispielsweise bei einer Ping-Latenz von über 200 ms oder einem VPN-Ausfall, automatisiert eine E-Mail-Benachrichtigung versendet.

Diese Pflichtziele decken die für das Vorhaben zentralen Funktionsebenen ab. Sie stellen sicher, dass sowohl auf Netzwerk- als auch auf Anwendungsebene relevante Zustände beobachtet werden können und dass die erhobenen Informationen nicht nur gesammelt,

1 Einleitung

sondern auch praktisch nutzbar gemacht werden. Die Kombination aus Datenerhebung, Speicherung, Visualisierung und Alarmierung bildet damit den funktionalen Kern des Prototyps.

1.3.2 Optionale Erweiterungen

Die optionalen Erweiterungen gehen über den definierten Mindestumfang hinaus und dienen dazu, mögliche Ausbaustufen des Systems aufzuzeigen. Sie sind insbesondere dann relevant, wenn einzelne Kernfunktionen früher als geplant umgesetzt werden oder wenn sich im Projektverlauf zusätzlicher Spielraum ergibt.

- **Ansible-Integration:** Automatisierte Bereitstellung der Monitoring-Infrastruktur auf mehreren Servern.
- **Zusätzliche Dashboards:** Getrennte Ansichten für Anwendungs-, Netzwerk- und Host-Ebene.
- **Erweiterte Alarmregeln:** Mehrkriterielle Alarmierung, etwa bei gleichzeitiger Prüfung von VPN-Status, Ping-Latenz und Port-Erreichbarkeit.
- **Rollen- und Rechte management:** Verwaltung von Benutzerrollen und Zugriffsrechten innerhalb des Grafana-Dashboards.

Die optionalen Erweiterungen verdeutlichen, dass das entworfene System grundsätzlich skalierbar und ausbaufähig angelegt ist. Gleichzeitig unterstreichen sie, dass die vorliegende Arbeit einen bewussten Fokus auf einen belastbaren Kernumfang legt, ohne weiterführende Entwicklungsmöglichkeiten auszuschließen.

1.4 Geplantes Vorgehen

Das Vorgehen orientiert sich an einem typischen Ablauf informatischer Abschlussarbeiten. Zunächst wird der fachliche und organisatorische Rahmen präzisiert, insbesondere im Hinblick auf die zu überwachenden Hosts, Netzwerkpfade und VPN-Verbindungen, geeignete Schwellenwerte sowie datenschutzbezogene Anforderungen. Diese Präzisierung ist notwendig, um ein realistisches und konsistentes Zielsystem für die weiteren Projektphasen festzulegen.

Anschließend erfolgt eine Recherche zu Spring Boot, Micrometer, Prometheus und Grafana, um die Eignung dieser Technologien für das geplante Monitoring-Szenario zu bewerten. Dabei geht es nicht nur um eine funktionale Betrachtung einzelner Werkzeuge, sondern auch um deren Zusammenspiel innerhalb einer Gesamtarchitektur. Die Recherche dient damit zugleich als Grundlage für spätere Technologieentscheidungen.

Darauf aufbauend wird ein Systementwurf entwickelt, der Architektur und Datenflüsse der Lösung beschreibt. Dieser Entwurf legt fest, welche Komponenten miteinander kommunizieren, wie Metriken erzeugt und übertragen werden und an welchen Stellen Visualisierung sowie Alarmierung ansetzen. Eine saubere konzeptionelle Vorarbeit ist wesentlich, damit die spätere Implementierung nicht nur funktioniert, sondern auch nachvollziehbar strukturiert ist.

Danach erfolgt die prototypische Implementierung, vorzugsweise in einer containerisierten Umgebung mit Docker. Die Containerisierung erleichtert eine reproduzierbare Bereitstellung der einzelnen Komponenten und unterstützt damit sowohl die technische

Umsetzung als auch die spätere Demonstration des Systems. In diesem Schritt wird geprüft, ob sich die zuvor entworfene Architektur mit den ausgewählten Technologien konsistent umsetzen lässt.

Abschließend wird das System anhand typischer Fehlerszenarien wie erhöhter Ping-Latenz, geschlossenen Ports oder eines VPN-Ausfalls getestet und hinsichtlich Erkennungszeit, Dashboard-Qualität und Alarmierungsfunktion bewertet. Die Evaluation dient nicht nur dazu, die Funktionalität des Prototyps nachzuweisen, sondern auch seine Grenzen sichtbar zu machen und mögliche Ansatzpunkte für spätere Erweiterungen zu identifizieren.

1.5 Aufbau der Arbeit

Nach der Einleitung werden zunächst die für das Verständnis der Arbeit erforderlichen fachlichen und technischen Grundlagen dargestellt. Anschließend folgt die Analyse des Problemraums, der Anforderungen sowie relevanter bestehender Ansätze.

Darauf aufbauend beschreibt die Arbeit die Konzeption des entwickelten Systems und erläutert anschließend dessen prototypische Implementierung. Abschließend erfolgt eine Evaluation der Lösung, bevor die zentralen Ergebnisse zusammengefasst und mögliche Erweiterungen im Ausblick dargestellt werden.

2 Grundlage

2.1 Grundbegriffe und Definitionen

Die folgenden Ausführungen führen die zentralen Begriffe ein, die für das Verständnis der im Rahmen dieser Arbeit entwickelten Monitoring-Lösung erforderlich sind. Sie schaffen ein einheitliches Begriffsverständnis als Grundlage für die nachfolgenden Kapitel zur Analyse, Konzeption und Implementierung.

2.1.1 Netzwerk-Monitoring, Verfügbarkeit und Latenz

Unter Netzwerk-Monitoring wird die kontinuierliche Überwachung des Zustands und der Leistungsfähigkeit von Netzwerkkomponenten, Diensten und Schnittstellen verstanden. Dazu gehören beispielsweise Server, Kommunikationsdienste und Webanwendungen, die über ein Netzwerk erreichbar sein müssen. Monitoring-Systeme erfassen hierzu Daten über Verfügbarkeit, Antwortzeiten und Fehlersituationen, um Störungen frühzeitig zu erkennen und geeignete Maßnahmen einleiten zu können [4].

Die Verfügbarkeit eines Dienstes beschreibt, in welchem Anteil einer betrachteten Zeitspanne dieser seinen vorgesehenen Zweck erfüllen kann. Sie wird häufig als Prozentwert angegeben, beispielsweise als „99,9 % Verfügbarkeit“. Für die operative Überwachung ist zusätzlich die Erreichbarkeit zu einem konkreten Zeitpunkt relevant. Ein Ziel gilt als erreichbar, wenn eine Verbindung innerhalb eines vorgegebenen Zeitlimits aufgebaut werden kann. Schlägt der Verbindungsaufbau fehl oder wird das Zeitlimit überschritten, wird das Ziel als nicht erreichbar bewertet.

Neben der Erreichbarkeit ist die Latenz eine wichtige Kennzahl zur Beurteilung der Dienstqualität. Sie beschreibt die Zeitspanne zwischen dem Start einer Anfrage und dem Eintreffen einer zugehörigen Antwort beziehungsweise eines Verbindungsstatus. Im Netzwerkumfeld wird sie üblicherweise in Millisekunden angegeben. Eine erhöhte Latenz kann auf Überlastung, Netzwerkprobleme oder eine eingeschränkte Leistungsfähigkeit eines Dienstes hinweisen.

2.1.2 Metrikbasiertes Monitoring und Alerting

Eine Metrik ist eine messbare Kennzahl, die einen bestimmten Aspekt des Zustands oder der Leistungsfähigkeit eines Systems beschreibt. Beispiele sind CPU-Auslastung, Arbeitsspeicherkonsum, Anzahl von Anfragen oder Latenzwerte. Bei einem metrikbasierten Monitoring werden diese Kennzahlen in regelmäßigen Abständen erfasst und als Zeitreihen gespeichert. Dadurch können aktuelle Zustände bewertet sowie Veränderungen über einen längeren Zeitraum analysiert werden [4].

Ein Exporter stellt Metriken in einem standardisierten Format über einen Endpunkt bereit. Ein Monitoring-System kann diese Daten anschließend abholen und speichern. Prometheus verwendet hierfür ein Pull- bzw. Scraping-Modell. Dabei ruft der Prometheus-Server die Metrik-Endpunkte der konfigurierten Ziele in festgelegten Intervallen ab [23].

Alerting bezeichnet die automatische Erkennung und Meldung kritischer Zustände auf Grundlage von Metriken. Eine Alert-Regel definiert eine Bedingung, bei deren Erfüllung eine Meldung erzeugt wird. Beispiele sind die Nichterreichbarkeit eines Dienstes oder

das Überschreiten eines Latenzgrenzwertes. Durch Alerting können Verantwortliche über relevante Störungen informiert werden, ohne ein Dashboard kontinuierlich überwachen zu müssen [21].

2.1.3 Containerisierung und TCP-basierte Erreichbarkeitsprüfung

Container sind isolierte Laufzeitumgebungen, in denen Anwendungen zusammen mit ihren Abhängigkeiten ausgeführt werden. Im Vergleich zu virtuellen Maschinen teilen Container den Kernel des Host-Betriebssystems und können daher mit geringerem Ressourcenaufwand gestartet und betrieben werden. Ein Container-Image dient dabei als Vorlage, aus der ein oder mehrere gleichartige Container erzeugt werden können.

Die gemeinsame Verwaltung mehrerer Container wird als Container-Orchestrierung bezeichnet. Sie umfasst unter anderem die Konfiguration von Diensten, Netzwerken, Volumes und Abhängigkeiten. Docker Compose ermöglicht die deklarative Definition und Ausführung von Anwendungen, die aus mehreren Containern bestehen, mithilfe einer YAML-Datei [7].

Die TCP-basierte Erreichbarkeitsprüfung dient der Überwachung eines konkreten Dienstes auf einem definierten Host und Port. Das Transmission Control Protocol (TCP) ist verbindungsorientiert und setzt vor einer Datenübertragung einen Verbindungsaufbau zwischen Client und Server voraus. Kann eine TCP-Verbindung erfolgreich aufgebaut werden, ist der Dienst auf dem angegebenen Port grundsätzlich erreichbar. Ein fehlgeschlagener Verbindungsaufbau weist dagegen auf einen nicht verfügbaren Dienst, ein Netzwerkproblem oder eine blockierende Netzwerkkonfiguration hin.

2.2 Technologische Grundlagen

Die entwickelte Monitoring-Lösung basiert auf mehreren miteinander verbundenen Technologien. Spring Boot bildet die Grundlage des Monitoring-Dienstes. Micrometer, Prometheus, Grafana und Alertmanager übernehmen die Erfassung, Speicherung, Visualisierung und Verarbeitung von Monitoring-Daten. Docker und Docker Compose ermöglichen die reproduzierbare Bereitstellung der beteiligten Komponenten.

2.2.1 Spring Boot, Actuator und Micrometer

Spring Boot ist ein Framework zur Entwicklung Java-basierter Anwendungen. Es vereinfacht die Konfiguration typischer Komponenten und unterstützt unter anderem die Bereitstellung von Webschnittstellen und Betriebsfunktionen. Das Actuator-Modul stellt produktionsnahe Funktionen und HTTP-Endpunkte für Zustandsinformationen, Health-Informationen und Metriken bereit [26].

Micrometer ist eine Metrik-Fassade für Java-Anwendungen. Die Bibliothek ermöglicht die Erfassung von Messwerten unabhängig vom konkret eingesetzten Monitoring-Backend. Spring Boot Actuator unterstützt Micrometer durch automatische Konfiguration und kann Metriken für unterschiedliche Monitoring-Systeme bereitstellen [26].

Für Prometheus stellt Spring Boot bei aktivierter Prometheus-Registry den Endpunkt `/actuator/prometheus` bereit. Dieser Endpunkt liefert Metriken im Prometheus-kompatiblen Format und kann regelmäßig durch einen Prometheus-Server abgefragt werden [26]. In der entwickelten Anwendung dient dieser Mechanismus dazu, selbst erzeugte Status- und Latenzmetriken für die weitere Verarbeitung bereitzustellen.

2.2.2 Prometheus, Grafana und Alertmanager

Prometheus ist ein Monitoring-System, das Metriken von konfigurierten Zielen abrufen und als Zeitreihen speichert. Die Konfiguration definiert unter anderem Abrufintervalle, Zielsysteme, Metrikpfade und Alert-Regeln. Auf Grundlage der gespeicherten Zeitreihen können Abfragen durchgeführt, Visualisierungen erstellt und kritische Zustände erkannt werden [23].

Im entwickelten System fragt Prometheus den Actuator-Endpunkt der Spring-Boot-Anwendung in festen Intervallen ab. Die Anwendung stellt dabei unter anderem die Metriken `network_device_up` und `network_device_latency_ms` bereit. Diese Werte werden von Prometheus als Zeitreihen gespeichert und können anschließend durch Grafana oder durch Alert-Regeln ausgewertet werden.

Grafana dient der Visualisierung der in Prometheus gespeicherten Daten. Prometheus wird dabei als Datenquelle konfiguriert. Grafana ermöglicht die Darstellung von Zeitreihen, Tabellen und Statusanzeigen, wodurch beispielsweise Verfügbarkeit und Latenz einzelner Netzwerkziele nachvollziehbar werden [9].

Alertmanager verarbeitet Alerts, die von Prometheus aufgrund definierter Regeln erzeugt werden. Zu seinen zentralen Aufgaben zählen die Deduplizierung, Gruppierung und Weiterleitung von Meldungen an konfigurierte Empfänger [22]. In der vorliegenden Lösung können Regeln beispielsweise einen Alert auslösen, wenn ein Ziel über einen definierten Zeitraum nicht erreichbar ist oder ein festgelegter Latenzgrenzwert überschritten wird.

2.2.3 Containerisierte Bereitstellung mit Docker und Docker Compose

Die Monitoring-Lösung wird mit Docker und Docker Compose als mehrteilige Containeranwendung bereitgestellt. Docker Compose ermöglicht es, Services, Netzwerke und Volumes in einer gemeinsamen YAML-Datei zu beschreiben und als zusammengehörigen Anwendungsstack zu starten [7].

Die Anwendung besteht aus getrennten Containern für den Spring-Boot-Monitoring-Dienst, Prometheus, Grafana und Alertmanager. Ein gemeinsames Docker-Netzwerk ermöglicht die Kommunikation der Dienste über ihre Servicenamen. Beispielsweise kann Prometheus den Monitoring-Dienst über den Hostnamen `app` erreichen, während Grafana Prometheus über den Hostnamen `prometheus` als Datenquelle verwendet.

Für Komponenten mit dauerhaft benötigten Daten, insbesondere Prometheus und Grafana, können Volumes verwendet werden. Damit bleiben Zeitreihendaten und lokale Einstellungen auch nach einem Neustart einzelner Container erhalten. Die containerisierte Bereitstellung reduziert den manuellen Installationsaufwand und erleichtert eine reproduzierbare Ausführung der gesamten Monitoring-Umgebung.

2.3 Verwendete Methoden und Konzepte des entwickelten Systems

Die entwickelte Monitoring-Lösung folgt mehreren methodischen und architektonischen Prinzipien. Im Mittelpunkt stehen eine konfigurationsbasierte Verwaltung der Überwachungsziele, die regelmäßige Überprüfung ihrer Erreichbarkeit sowie die Bereitstellung der Ergebnisse als Metriken. Ergänzend wurden eine klar getrennte Schichtenarchitektur, eine robuste Fehlerbehandlung und automatisierte Tests für zentrale Funktionen umgesetzt.

2.3.1 Konfigurationsgetriebenes Gerätemanagement und zeitgesteuertes Polling

Die zu überwachenden Netzwerkziele werden nicht fest im Quellcode hinterlegt, sondern über die Anwendungskonfiguration definiert. Zu jedem Ziel werden ein Name, ein Host, ein Port sowie ein Aktivierungsstatus angegeben. Zusätzlich werden zentrale Parameter wie das Prüfintervall und das Verbindungs-Timeout konfiguriert.

Die Konfiguration wird beim Start der Anwendung in eine strukturierte Konfigurationsklasse übernommen. Dadurch wird die technische Beschreibung der Überwachungsziele von der eigentlichen Prüfungslogik getrennt. Neue Ziele können durch die Anpassung der Konfigurationsdatei ergänzt werden, ohne dass Änderungen an den Java-Klassen erforderlich sind. Der Aktivierungsstatus ermöglicht außerdem, einzelne Ziele bei Bedarf vorübergehend von der Überwachung auszunehmen.

Die Überwachung erfolgt nach dem Prinzip des zeitgesteuerten Pollings. Dabei überprüft der Monitoring-Dienst alle aktivierten Ziele in regelmäßigen Abständen. Für die Umsetzung wird die Scheduling-Funktion des Spring Framework verwendet. Methoden, die mit der Annotation `@Scheduled` versehen sind, können in festgelegten Zeitintervallen automatisiert ausgeführt werden [27].

Das Polling-Verfahren stellt einen Kompromiss zwischen Aktualität und Ressourcenverbrauch dar. Kürzere Intervalle ermöglichen eine schnellere Erkennung von Störungen, verursachen jedoch mehr Verbindungsversuche und Metrikaktualisierungen. Längere Intervalle reduzieren den Aufwand, führen jedoch zu einer größeren Verzögerung bei der Erkennung eines Ausfalls. Durch die Konfigurierbarkeit des Intervalls kann dieser Kompromiss an die jeweilige Einsatzumgebung angepasst werden.

2.3.2 Zustandsbasierte Metriken und Kennzahlenmodell

Die Ergebnisse der Erreichbarkeitsprüfungen werden als zustandsbasierte Metriken modelliert. Für jedes überwachte Ziel werden zwei Kennzahlen bereitgestellt: ein Verfügbarkeitsstatus und eine Latenzmetrik. Der Status wird durch die Metrik `network_device_up` dargestellt. Ein Wert von 1 bedeutet, dass der Verbindungsaufbau erfolgreich war; ein Wert von 0 kennzeichnet ein nicht erreichbares Ziel.

Die zweite Metrik `network_device_latency_ms` enthält die Dauer des letzten Verbindungsversuchs in Millisekunden. Beide Werte werden als Gauges exportiert. Ein Gauge ist für diese Art von Information geeignet, da er einen aktuellen Zustand beschreibt und sich sowohl erhöhen als auch verringern kann. Dies unterscheidet ihn beispielsweise von einem Counter, der ausschließlich monoton ansteigt [24].

Zur Unterscheidung der einzelnen Ziele werden die Metriken mit den Labels `device` und `host` versehen. Dadurch können Prometheus-Abfragen, Alert-Regeln und Grafana-Dashboards den Status und die Latenz eines konkreten Ziels auswerten. Das Kennzahlenmodell bildet damit die Grundlage für die Visualisierung und die automatische Erkennung von Störungen.

2.3.3 Schichtenarchitektur, Fehlerbehandlung und Robustheit

Die Anwendung ist in mehrere fachlich getrennte Komponenten gegliedert. Diese Struktur folgt dem Prinzip der Schichtenarchitektur und unterstützt eine klare Zuordnung von Verantwortlichkeiten. Die Konfigurationskomponente übernimmt das Einlesen und Bereitstellen der Einstellungen aus der Datei `application.yml`. Das Datenmodell umfasst die Klassen `Device` und `CheckResult`. Während `Device` ein zu überwachendes Ziel

beschreibt, enthält `CheckResult` das Ergebnis einer konkreten Prüfung einschließlich Erreichbarkeitsstatus, Latenz und Prüfzeitpunkt.

Die zentrale fachliche Logik befindet sich im `DeviceCheckService`. Dieser Dienst führt die Prüfungen aus, aktualisiert die Metrikwerte und hält die zuletzt ermittelten Prüfergebnisse bereit. Der `MonitoringScheduler` stößt die periodische Ausführung an. Die Controller-Schicht stellt REST-Endpunkte bereit, über die Geräte, aktuelle Prüfergebnisse und der Status des Monitoring-Dienstes abgefragt werden können. Zusätzlich ist eine manuelle Auslösung eines Prüfzyklus vorgesehen.

Netzwerkprüfungen sind fehleranfällig, da Verbindungsprobleme, nicht verfügbare Dienste, ungültige Zieladressen oder Zeitüberschreitungen auftreten können. Deshalb werden Ausnahmen beim Aufbau einer TCP-Verbindung behandelt. Schlägt ein Verbindungsaufbau fehl, wird das betreffende Ziel als nicht erreichbar eingestuft; die Prüfung der weiteren Ziele wird dennoch fortgesetzt.

Ein konfigurierbares Timeout begrenzt die Wartezeit für jeden Verbindungsversuch. Dadurch wird verhindert, dass nicht reagierende Ziele den Prüfzyklus über einen längeren Zeitraum blockieren. Auch bei einem fehlgeschlagenen Verbindungsversuch werden Status und Dauer der Prüfung als Ergebnis gespeichert. Die Kombination aus Fehlerbehandlung, Timeout und unabhängiger Verarbeitung einzelner Ziele trägt zur Robustheit der Monitoring-Lösung bei.

2.3.4 Automatisierte Tests der Kernlogik

Zur Absicherung zentraler Funktionen wurden automatisierte Unit-Tests mit JUnit 5 umgesetzt. JUnit stellt einen Rahmen zur Definition und automatisierten Ausführung von Tests für Java-Anwendungen bereit [14]. Der Schwerpunkt liegt auf dem `DeviceCheckService`, da dieser die Konfiguration der Ziele verarbeitet und die Ergebnisse der Erreichbarkeitsprüfungen erzeugt.

Ein Test prüft, ob die in der Konfiguration definierten Geräte korrekt in die interne Geräteliste übernommen werden. Dadurch wird sichergestellt, dass die konfigurationsgetriebene Verwaltung der Ziele wie vorgesehen funktioniert. Ein weiterer Test verwendet ein nicht erreichbares Ziel und prüft, ob dieses als nicht erreichbar bewertet wird. Zusätzlich wird geprüft, ob für den Verbindungsversuch ein gültiger Latenzwert erzeugt wird.

Die Tests ermöglichen eine wiederholbare Überprüfung der Kernlogik, ohne dass dafür die vollständige Containerumgebung mit Prometheus, Grafana und Alertmanager gestartet werden muss. Sie reduzieren damit das Risiko, dass Änderungen an der Anwendung unbemerkt zu Fehlern in der grundlegenden Monitoring-Funktion führen.

2.4 Stand der Technik und Einordnung

Blackbox Exporter??? TBD

3 Analyse

Dieses Kapitel analysiert den Problemraum, ermittelt die Anforderungen an das zu entwickelnde Monitoring-System und untersucht den Stand der Technik sowie verwandte Arbeiten aus Forschung und Praxis. Aufbauend auf der Einleitung und den Grundlagen wird zunächst der aktuelle Zustand in typischen IT-Service-Umgebungen beschrieben, anschließend werden die Anforderungen systematisch abgeleitet und schließlich bestehende Monitoring-Lösungen hinsichtlich ihrer Abdeckung der ermittelten Anforderungen analysiert. Die identifizierten Lücken zwischen Anforderungen und bestehenden Lösungen begründen die Notwendigkeit der entwickelten Lösung.

3.1 Analyse des Problemraums

In vielen IT-Service-Abteilungen, insbesondere im Umfeld von Bildungseinrichtungen und mittelgroßen Organisationen, wird die Überwachung von Netzwerkverbindungen und Diensten weiterhin stark manuell betrieben. Typische Vorgehensweisen umfassen sporadische Ping-Tests, manuelle Überprüfungen von VPN-Endpunkten oder reaktive Fehleranalysen erst nach eingegangenen Benutzerbeschwerden.

Diese Vorgehensweise weist mehrere charakteristische Schwachstellen auf:

- **Verzögerte Erkennung:** Störungen werden oft erst mit erheblicher Verzögerung bemerkt, da keine kontinuierliche automatisierte Überwachung stattfindet.
- **Hoher manueller Aufwand:** Regelmäßige manuelle Tests binden personelle Ressourcen und sind nur stichprobenartig durchführbar.
- **Fehlende Historie:** Ohne systematische Speicherung von Messwerten können vergangene Zustände nicht zuverlässig rekonstruiert werden.
- **Keine automatisierte Alarmierung:** Bei Ausfällen erfolgt keine unmittelbare Benachrichtigung verantwortlicher Personen.
- **Eingeschränkte Nachvollziehbarkeit:** Zusammenhänge zwischen verschiedenen Fehlerbildern lassen sich im Nachhinein nur schwer analysieren.

Diese Probleme treten insbesondere dann verstärkt auf, wenn mehrere Systeme voneinander abhängen und Störungen kettenartig auf andere Dienste wirken. Für einen professionellen IT-Betrieb ist jedoch nicht nur die unmittelbare Störungserkennung relevant, sondern auch die Möglichkeit, Entwicklungen über einen längeren Zeitraum nachzuvollziehen, Grenzwerte zu definieren und daraus gezielte Maßnahmen abzuleiten.

3.2 Anforderungsermittlung

Aus der Problemstellung und den Zielen der Arbeit ergeben sich funktionale und nicht-funktionale Anforderungen an das zu entwickelnde Monitoring-System. Die Anforderungen wurden durch eine Kombination aus Problemraumanalyse, Zieldefinition und Abgleich mit bestehenden Lösungen ermittelt.

3.2.1 Funktionale Anforderungen

Die zentralen funktionalen Anforderungen lassen sich wie folgt zusammenfassen:

- **F1 Konfigurierbare Überwachungsziele:** Das System muss in der Lage sein, eine Liste von Netzwerkzielen zu verwalten. Jedes Ziel umfasst mindestens einen Namen, einen Hostnamen oder eine IP-Adresse, einen TCP-Port und einen Aktivierungsstatus.
- **F2 Regelmäßige Erreichbarkeitsprüfung:** Alle aktivierten Ziele müssen in konfigurierbaren Zeitintervallen automatisch geprüft werden.
- **F3 Ermittlung von Status und Latenz:** Für jedes Ziel muss der Erreichbarkeitsstatus sowie die Dauer des Verbindungsversuchs ermittelt werden.
- **F4 Bereitstellung als Metriken:** Die Ergebnisse müssen in einem standardisierten Format bereitgestellt werden, das von Prometheus konsumiert werden kann.
- **F5 Zeitreihenspeicherung:** Die Metriken müssen von Prometheus als Zeitreihen gespeichert werden, um aktuelle und historische Werte auswerten zu können.
- **F6 Visualisierung:** Ein Dashboard muss Erreichbarkeit, Latenz sowie ausgewählte Anwendungsmetriken darstellen können.
- **F7 Alert-Erkennung:** Das System muss in der Lage sein, definierte Fehlerzustände automatisch zu erkennen und als Alert zu melden.
- **F8 Incident-Historie:** Alerts müssen in einer Incident-Historie gespeichert werden, um vergangene Störungen nachvollziehbar zu machen.
- **F9 REST-Schnittstelle:** Das System muss Endpunkte bereitstellen, um konfigurierte Geräte, aktuelle Prüfergebnisse, Incident-Historie und den Dienststatus abzufragen.
- **F10 Manuelle Auslösung:** Ein Prüfzyklus muss auch manuell ausgelöst werden können, um Tests und Demonstrationen zu unterstützen.

3.2.2 Nichtfunktionale Anforderungen

Neben den fachlichen Funktionen gelten folgende Qualitätsanforderungen:

- **N1 Konfigurierbarkeit:** Überwachungsziele, Prüfintervalle und Timeouts sollen ohne Änderungen an der Java-Fachlogik angepasst werden können.
- **N2 Robustheit:** Der Ausfall eines einzelnen Ziels darf nicht zum Abbruch des gesamten Prüfzyklus führen.
- **N3 Reproduzierbarkeit:** Die Komponenten sollen mit Docker Compose einheitlich gestartet und konfiguriert werden können.
- **N4 Erweiterbarkeit:** Neue Prüffarten und zusätzliche Metriken sollen in die bestehende Struktur integriert werden können.
- **N5 Nachvollziehbarkeit:** Zustände und Messwerte sollen über Zeitreihen und Dashboard-Ansichten analysierbar sein.

- **N6 Ressourceneffizienz:** Prüfintervall und Timeout sollen so gewählt beziehungsweise konfiguriert werden können, dass Aktualität und Systemlast in einem angemessenen Verhältnis stehen.
- **N7 Sicherheit:** Administrative Endpunkte müssen durch Authentifizierung und Autorisierung geschützt sein.
- **N8 Leichtgewichtigkeit:** Die Lösung soll bewusst schlank gehalten sein und sich auf die Kernfunktionen konzentrieren.

3.3 Stand der Technik und verwandte Arbeiten

Im Bereich Netzwerk-Monitoring existieren zahlreiche etablierte Werkzeuge und Plattformen. Die folgende Analyse betrachtet ausgewählte Lösungen aus Forschung und Praxis und ordnet diese im Kontext der ermittelten Anforderungen ein.

3.3.1 Prometheus

Prometheus ist ein Open-Source-Monitoring-System, das Metriken von konfigurierten Zielen abrufen und als Zeitreihen speichern. Die Konfiguration definiert unter anderem Abrufintervalle, Zielsysteme, Metrikpfade und Alert-Regeln. Auf Grundlage der gespeicherten Zeitreihen können Abfragen durchgeführt, Visualisierungen erstellt und kritische Zustände erkannt werden [5].

Prometheus verwendet ein Pull- beziehungsweise Scraping-Modell. Der Prometheus-Server ruft die Metrik-Endpunkte der konfigurierten Ziele in festgelegten Intervallen ab. Prometheus deckt die Anforderungen F4, F5 und F7 ab, bietet jedoch keine integrierte Incident-Historie (F8) und keine REST-Schnittstelle für operative Abfragen (F9).

3.3.2 Prometheus Blackbox Exporter

Der Prometheus Blackbox Exporter ermöglicht die Überwachung von Endpunkten über Protokolle wie HTTP, HTTPS, DNS, TCP und ICMP. Er führt aktive Prüfungen durch und stellt die Ergebnisse als Metriken bereit.

Der Blackbox Exporter deckt die Anforderungen F1, F2, F3 und F4 ab. Allerdings fehlen auch hier eine Incident-Historie (F8) und eine REST-Schnittstelle (F9). Die Konfiguration erfolgt über YAML-Dateien, was N1 teilweise erfüllt, jedoch keine dynamische Verwaltung ermöglicht.

3.3.3 Grafana

Grafana dient der Visualisierung von Zeitreihendaten aus verschiedenen Quellen, darunter Prometheus. Grafana ermöglicht die Darstellung von Zeitreihen, Tabellen und Statusanzeigen, wodurch beispielsweise Verfügbarkeit und Latenz einzelner Netzwerkziele nachvollziehbar werden [17].

Grafana deckt die Anforderung F6 ab, ist jedoch auf externe Datenquellen angewiesen und bietet keine eigene Datenerfassung oder Incident-Historie.

3.3.4 Alertmanager

Alertmanager verarbeitet Alerts, die von Prometheus aufgrund definierter Regeln erzeugt werden. Zu seinen zentralen Aufgaben zählen die Deduplizierung, Gruppierung und Weiterleitung von Meldungen an konfigurierte Empfänger [2].

Alertmanager deckt Teile von F7 ab, bietet jedoch keine persistente Incident-Historie (F8) und keine REST-Schnittstelle für externe Abfragen (F9).

3.3.5 Kommerzielle und Open-Source-Monitoring-Plattformen

Neben Prometheus existieren weitere etablierte Monitoring-Lösungen wie Nagios, Zabbix oder Icinga. Diese Werkzeuge bieten umfassende Funktionen für Infrastruktur-Monitoring, Alerting und Reporting.

Nagios ist eine weit verbreitete Open-Source-Monitoring-Lösung, die umfangreiche Prüfungen und Benachrichtigungen unterstützt. Allerdings ist die Konfiguration komplex und die Architektur monolithisch. Nagios bietet keine native Integration mit Prometheus und keine moderne REST-API.

Zabbix ist eine Enterprise-Monitoring-Plattform mit umfassenden Funktionen für Netzwerk-, Server- und Anwendungsmonitoring. Zabbix bietet eine Datenbank-basierte Historie und eine REST-API, ist jedoch in der Einrichtung und Wartung aufwendig und für leichtgewichtige Szenarien überdimensioniert.

Icinga ist eine Weiterentwicklung von Nagios mit verbesserter Skalierbarkeit und einer moderneren Architektur. Auch Icinga bietet umfangreiche Funktionen, ist jedoch ebenfalls komplex in der Konfiguration und erfordert erheblichen Wartungsaufwand.

Diese Lösungen decken viele der funktionalen Anforderungen ab (F1–F7), sind jedoch oft zu komplex für leichtgewichtige Szenarien (N8) und bieten keine einfache containerisierte Bereitstellung (N3).

3.3.6 Forschungsarbeiten

Im Bereich Monitoring und Observability existieren zahlreiche Forschungsarbeiten, die sich mit Themen wie automatischer Fehlererkennung, Metrikanalyse und Alert-Optimierung beschäftigen. Für die vorliegende Arbeit sind insbesondere Arbeiten relevant, die sich mit leichtgewichtigen Monitoring-Architekturen und Incident-Management befassen.

Da der Fokus dieser Arbeit auf einer praxisnahen Implementierung liegt, werden Forschungsarbeiten nicht im Detail analysiert. Die entwickelten Konzepte orientieren sich jedoch an etablierten Praktiken aus Forschung und Industrie, insbesondere im Bereich Metrik-basiertes Monitoring und Incident-Management.

3.3.7 Abdeckungsanalyse und identifizierte Lücken

Tabelle 3.1 zeigt eine vergleichende Analyse der ermittelten Anforderungen im Kontext bestehender Lösungen.

Die Analyse zeigt folgende identifizierte Lücken:

- **Lücke 1: Fehlende Incident-Historie:** Weder Prometheus noch Blackbox Exporter noch typische Nagios/Zabbix-Installationen bieten eine integrierte, leicht zugängliche Incident-Historie. Dies erschwert die langfristige Nachverfolgung von Störungen.
- **Lücke 2: Fehlende integrierte REST-Schnittstelle:** Bestehende Lösungen bieten keine umfassende REST-Schnittstelle für operative Abfragen von Geräten, Prüfergebnissen und Incidents.
- **Lücke 3: Komplexe Bereitstellung:** Viele bestehende Lösungen sind nicht als integrierte, containerisierte Anwendung bereitstellbar, was die Reproduzierbarkeit und Wartung erschwert.

Tabelle 3.1: Vergleichende Anforderungsabdeckung bestehender Monitoring-Lösungen

Anforderung	Prometheus	Blackbox	Nagios/Zabbix	Eigene Lösung
F1 Konfigurierbare Ziele	Ja	Ja	Ja	Ja
F2 Regelmäßige Prüfung	Ja	Ja	Ja	Ja
F3 Status und Latenz	Ja	Ja	Ja	Ja
F4 Metrik-Export	Ja	Ja	Nein	Ja
F5 Zeitreihenspeicherung	Ja	Nein	Ja	Ja
F6 Visualisierung	Nein ¹	Nein	Ja ²	Ja ³
F7 Alert-Erkennung	Ja	Nein	Ja	Ja
F8 Incident-Historie	Nein	Nein	Nein ⁴	Ja
F9 REST-Schnittstelle	Nein	Nein	Nein ⁵	Ja
F10 Manuelle Auslösung	Nein	Nein	Nein	Ja
N1 Konfigurierbarkeit	Ja	Ja	Nein	Ja
N2 Robustheit	Ja	Ja	Ja	Ja
N3 Reproduzierbarkeit	Nein ⁶	Nein ⁶	Nein	Ja
N4 Erweiterbarkeit	Ja	Ja	Nein	Ja
N5 Nachvollziehbarkeit	Ja	Ja	Ja	Ja
N6 Ressourceneffizienz	Ja	Ja	Nein	Ja
N7 Sicherheit	Ja ⁷	Ja ⁷	Ja	Ja
N8 Leichtgewichtigkeit	Ja	Ja	Nein	Ja

¹ Benötigt Grafana als separate Komponente² Benötigt zusätzliche Plugins³ Über Grafana und eigenes Web-Dashboard⁴ Nur über externe Plugins oder Custom-Lösungen⁵ Nur über externe APIs oder Plugins⁶ Docker-Support vorhanden, aber keine integrierte Compose-Bereitstellung⁷ Erfordert zusätzliche Konfiguration

- **Lücke 4: Überdimensionierung:** Umfassende Monitoring-Plattformen wie Nagios oder Zabbix sind für leichtgewichtige Szenarien oft überdimensioniert und erfordern erheblichen Konfigurationsaufwand.
- **Lücke 5: Fehlende manuelle Auslösung:** Die Möglichkeit, Prüfzyklen manuell auszulösen, ist in bestehenden Lösungen nicht standardmäßig vorhanden.

Diese Lücken begründen die Notwendigkeit der entwickelten Lösung. Die eigene Implementierung schließt diese Lücken durch eine integrierte Incident-Historie, eine umfassende REST-Schnittstelle, eine containerisierte Bereitstellung und eine bewusst leichtgewichtige Architektur.

3.4 Zusammenfassung

Die Analyse des Problemraums hat gezeigt, dass manuelle und unregelmäßige Netzwerkprüfungen in vielen IT-Service-Umgebungen zu verzögerter Störungserkennung, hohem manuellem Aufwand und eingeschränkter Nachvollziehbarkeit führen. Die abgeleiteten

3 Analyse

funktionalen und nichtfunktionalen Anforderungen bilden die Grundlage für die Konzeption und Implementierung des Monitoring-Systems.

Der Vergleich mit bestehenden Monitoring-Lösungen hat verdeutlicht, dass Prometheus, Grafana und Alertmanager eine geeignete Basis für Speicherung, Visualisierung und Alerting bieten, jedoch keine integrierte Incident-Historie und keine umfassende REST-Schnittstelle bieten. Kommerzielle Lösungen wie Nagios oder Zabbix sind für leichtgewichtige Szenarien überdimensioniert und erfordern erheblichen Konfigurationsaufwand.

Die identifizierte Lücke zwischen Anforderungen und bestehenden Lösungen begründet die Notwendigkeit der entwickelten Lösung. Durch die Kombination aus Spring-Boot-basierter Prüfkompone, Prometheus, Grafana, Alertmanager und einer eigenen Incident-Historie entsteht eine modulare, containerisierte und gut nachvollziehbare Monitoring-Architektur, die sich gezielt auf die Anforderungen der Arbeit konzentriert und bestehende Lücken schließt.

4 Konzeption

Dieses Kapitel beschreibt die Konzeption der entwickelten Monitoring-Lösung. Aufbauend auf den in Kapitel 3 ermittelten Anforderungen und den identifizierten Lücken werden eigene Lösungsansätze vorgestellt, die diese Lücken schließen und die Anforderungen erfüllen. Die Konzeption erfolgt technologieunabhängig, das heißt, die beschriebenen Konzepte und Architekturen sind unabhängig von konkreten Implementierungstechnologien wie Java, Spring Boot oder Prometheus formuliert.

4.1 Überblick über die Gesamtarchitektur

Die entwickelte Lösung folgt einer modularen Architektur, die aus mehreren miteinander verbundenen Komponenten besteht. Die Architektur orientiert sich an etablierten Prinzipien der Softwarearchitektur, insbesondere am Separation-of-Concerns-Prinzip und an der Schichtenarchitektur [25]. Abbildung 4.1 zeigt eine konzeptionelle Übersicht der Gesamtarchitektur.

Die Architektur umfasst folgende Hauptkomponenten:

- **Monitoring-Dienst:** Zentrale Komponente zur Verwaltung der Überwachungsziele und Ausführung der Erreichbarkeitsprüfungen.
- **Metrik-Export:** Bereitstellung der Prüfergebnisse in einem standardisierten Format für externe Monitoring-Systeme.
- **Zeitreihenspeicher:** Persistierung der Metriken als Zeitreihen für aktuelle und historische Auswertungen.
- **Visualisierung:** Darstellung der Metriken und Zustände in Dashboards.
- **Alert-Komponente:** Erkennung kritischer Zustände und Weiterleitung von Benachrichtigungen.
- **Incident-Speicher:** Persistente Speicherung von Alerts als Incidents zur langfristigen Nachverfolgung.
- **REST-Schnittstelle:** Operative Abfrage von Konfiguration, Prüfergebnissen und Incidents.

Die Komponenten kommunizieren über definierte Schnittstellen miteinander. Der Monitoring-Dienst führt die Prüfungen durch und stellt die Ergebnisse über den Metrik-Export bereit. Der Zeitreihenspeicher ruft die Metriken regelmäßig ab und speichert sie. Die Visualisierungskomponente greift auf den Zeitreihenspeicher zu, um Dashboards zu erstellen. Die Alert-Komponente wertet die gespeicherten Metriken aus und leitet bei kritischen Zuständen Benachrichtigungen an den Incident-Speicher weiter.

4 Konzeption

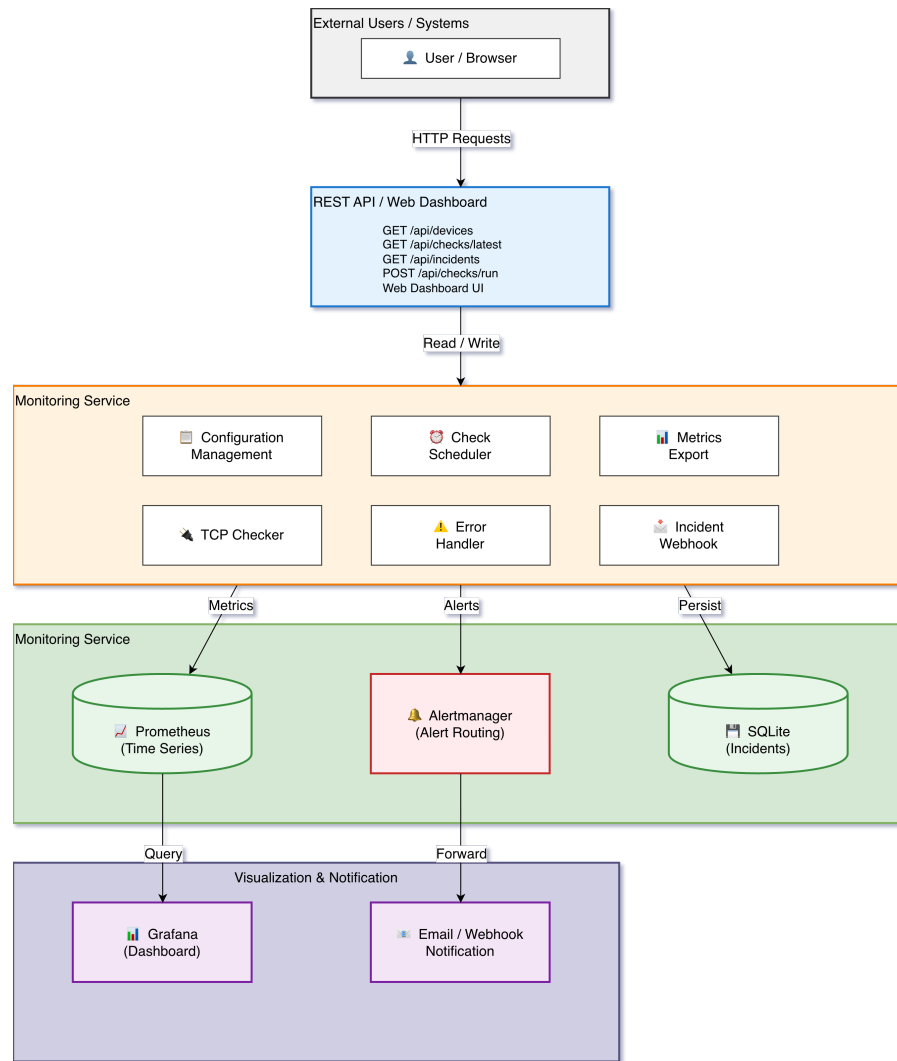


Abbildung 4.1: Konzeptionelle Übersicht der Gesamtarchitektur

4.2 Schließen der identifizierten Lücken

Die Konzeption adressiert explizit die in Abschnitt 3.3.7 identifizierten Lücken bestehender Monitoring-Lösungen. Die Lösungsansätze orientieren sich an bewährten Praktiken aus Forschung und Industrie [18].

4.2.1 Lücke 1: Fehlende Incident-Historie

Um die fehlende Incident-Historie bestehender Lösungen zu adressieren, wird eine eigene Komponente zur persistenten Speicherung von Incidents konzipiert. Jedes Incident umfasst folgende Informationen:

- Eindeutige Identifikation (z.B. Fingerprint des Alerts)
- Alert-Name und Beschreibung
- Betroffenes Gerät (Name, Host, Port)
- Schweregrad (z.B. Warning, Critical)

- Status (Firing oder Resolved)
- Startzeitpunkt und optionaler Auflösungszeitpunkt
- Zusammenfassung und detaillierte Beschreibung

Die Incident-Historie ermöglicht die langfristige Nachverfolgung von Störungen unabhängig von der Zeitreihen-Retention des Monitoring-Systems. Durch die Speicherung von Start- und Auflösungszeitpunkt kann die Dauer jeder Störung berechnet und analysiert werden. Dieses Konzept orientiert sich an etablierten Incident-Management-Praktiken [1].

4.2.2 Lücke 2: Fehlende integrierte REST-Schnittstelle

Um eine umfassende REST-Schnittstelle bereitzustellen, wird eine API-Komponente konzipiert, die folgende Endpunkte bereitstellt:

- Abfrage der konfigurierten Überwachungsziele
- Abfrage der aktuellen Prüfergebnisse
- Abfrage der historischen Prüfergebnisse mit Filtermöglichkeiten
- Abfrage aller Incidents und der aktiven Incidents
- Abfrage des Dienststatus
- Manuelle Auslösung eines Prüfzyklus
- Empfang von Alert-Webhooks

Die REST-Schnittstelle ermöglicht die Integration mit externen Systemen und die automatisierte Abfrage von Monitoring-Daten. Administrative Endpunkte werden durch Authentifizierung und Autorisierung geschützt. Die API-Design-Entscheidungen orientieren sich an den Prinzipien des REST-Architekturstils [8].

4.2.3 Lücke 3: Komplexe Bereitstellung

Um die Bereitstellung zu vereinfachen, wird die gesamte Monitoring-Umgebung als containerisierte Anwendung konzipiert. Alle Komponenten werden in separaten Containern ausgeführt und über ein gemeinsames Netzwerk verbunden. Die Konfiguration erfolgt über eine zentrale Konfigurationsdatei, die alle Services, Netzwerke und Volumes definiert.

Diese Konzeption ermöglicht:

- Einheitlichen Start aller Komponenten mit einem Befehl
- Reproduzierbare Bereitstellung in verschiedenen Umgebungen
- Einfache Skalierung und Wartung einzelner Komponenten
- Isolierte Ausführung mit definierten Ressourcenkontingenten

Die Containerisierung folgt etablierten Best Practices für containerisierte Anwendungen [12].

4.2.4 Lücke 4: Überdimensionierung

Um Überdimensionierung zu vermeiden, wird die Lösung bewusst schlank gehalten und konzentriert sich auf die Kernfunktionen:

- TCP-basierte Erreichbarkeitsprüfungen
- Metrik-Export für externe Monitoring-Systeme
- Visualisierung über externe Dashboards
- Incident-Management mit persistenter Historie
- REST-API für operative Abfragen

Komplexe Funktionen wie automatische Discovery, umfassende Plugin-Systeme oder Enterprise-Features werden bewusst nicht implementiert, um die Leichtgewichtigkeit zu erhalten. Dieses Vorgehen orientiert sich am Prinzip der minimalen Funktionalität (Minimal Viable Product) [20].

4.2.5 Lücke 5: Fehlende manuelle Auslösung

Um die manuelle Auslösung von Prüfzyklen zu ermöglichen, wird ein dedizierter Endpunkt konzipiert, der bei Aufruf einen sofortigen Prüfzyklus für alle aktivierten Ziele startet. Diese Funktion unterstützt Tests, Fehlersuche und Demonstrationen, da ein neuer Zustand gezielt erzeugt und unmittelbar überprüft werden kann.

4.3 Konzeption des Monitoring-Dienstes

Der Monitoring-Dienst bildet die zentrale Komponente der Lösung. Er übernimmt die Verwaltung der Überwachungsziele, die Ausführung der Prüfungen und die Bereitstellung der Ergebnisse.

4.3.1 Verwaltung der Überwachungsziele

Die Überwachungsziele werden in einer Konfigurationsdatei verwaltet. Jedes Ziel umfasst folgende Attribute:

- **Identifikation:** Eindeutige ID und lesbarer Name
- **Zieladresse:** Hostname oder IP-Adresse
- **Port:** Zu prüfender TCP-Port
- **Aktivierungsstatus:** Gibt an, ob das Ziel aktiv überwacht wird

Zusätzlich werden globale Parameter definiert:

- **Prüfintervall:** Zeit zwischen zwei Prüfzyklen
- **Timeout:** Maximale Wartezeit für einen Verbindungsversuch
- **Historien-Retention:** Speicherdauer für Prüfergebnisse und Incidents

Diese Trennung von Konfiguration und Fachlogik ermöglicht die Anpassung der Überwachungsziele ohne Änderungen am Quellcode (Anforderung N1). Dieses Konzept folgt dem Prinzip der Externalisierung von Konfigurationen [11].

4.3.2 Ausführung der Erreichbarkeitsprüfungen

Die Prüfungen erfolgen nach dem Prinzip des zeitgesteuerten Pollings. In regelmäßigen Intervallen wird für jedes aktivierte Ziel versucht, eine TCP-Verbindung zum konfigurierten Host und Port aufzubauen.

Abbildung 4.2 zeigt den Ablauf einer einzelnen Prüfung.

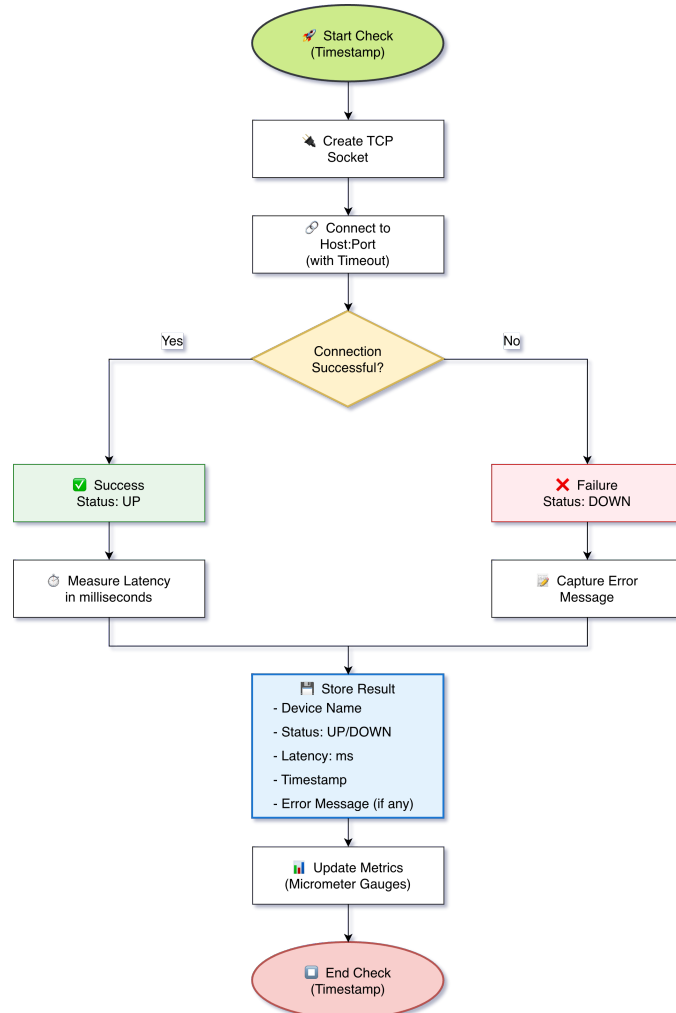


Abbildung 4.2: Ablauf einer Erreichbarkeitsprüfung

Der Prüfungsablauf umfasst folgende Schritte:

1. Start des Verbindungsversuchs mit Zeitstempel
2. Versuch, eine TCP-Verbindung zum Ziel aufzubauen
3. Bei Erfolg: Status auf „erreichbar“ setzen und Latenz messen
4. Bei Fehler: Status auf „nicht erreichbar“ setzen und Fehlermeldung speichern
5. Ende des Verbindungsversuchs mit Zeitstempel
6. Speichern des Ergebnisses für Metrik-Export und REST-Schnittstelle

Jedes Ziel wird unabhängig von den anderen Zielen geprüft. Der Ausfall eines einzelnen Ziels führt nicht zum Abbruch des gesamten Prüfzyklus (Anforderung N2).

4.3.3 Fehlerbehandlung und Robustheit

Die Fehlerbehandlung ist ein wesentlicher Bestandteil der Konzeption. Folgende Fehlerzenarien werden berücksichtigt:

- **Zeitüberschreitung:** Wenn ein Ziel nicht innerhalb des konfigurierten Timeouts antwortet, wird die Verbindung abgebrochen und das Ziel als nicht erreichbar markiert.
- **Netzwerkfehler:** Bei Netzwerkproblemen (z.B. kein Route zum Ziel) wird der Fehler protokolliert und das Ziel als nicht erreichbar markiert.
- **Ungültige Konfiguration:** Bei fehlerhaften Konfigurationseinträgen (z.B. ungültige IP-Adresse) wird das Ziel übersprungen und ein Fehler protokolliert.
- **Ressourcenfehler:** Bei Erschöpfung von Systemressourcen (z.B. keine freien Sockets) wird der Prüfzyklus unterbrochen und ein Fehler protokolliert.

Durch diese Maßnahmen wird die Robustheit des Systems sichergestellt (Anforderung N2). Die Fehlerbehandlung folgt etablierten Patterns für robuste Systeme [10].

4.4 Konzeption des Metrik-Exports

Der Metrik-Export stellt die Prüfergebnisse in einem standardisierten Format bereit, das von externen Monitoring-Systemen konsumiert werden kann.

4.4.1 Metrikmodell

Für jedes Überwachungsziel werden zwei zentrale Metriken bereitgestellt:

- **Status-Metrik:** Binärer Wert (1 für erreichbar, 0 für nicht erreichbar)
- **Latenz-Metrik:** Dauer des letzten Verbindungsversuchs in Millisekunden

Die Metriken werden mit Labels versehen, um die einzelnen Ziele zu unterscheiden. Vorgesehene Labels sind:

- **Name:** Lesbarer Name des Ziels
- **Host:** Hostname oder IP-Adresse
- **Port:** Geprüfter TCP-Port

Zusätzlich werden Anwendungsmetriken bereitgestellt, die den Zustand des Monitoring-Dienstes selbst beschreiben (z.B. Anzahl der konfigurierten Ziele, Anzahl der aktiven Ziele, letzte Prüfungszeit).

4.4.2 Exportformat

Das Exportformat orientiert sich an etablierten Standards für Metrik-Formate, insbesondere am Prometheus-Textformat [3]. Jede Metrik umfasst:

- Metrik-Name
- Labels (in geschweiften Klammern)

- Numerischer Wert
- Optionaler Hilfstext (Beschreibung der Metrik)

Beispielhafte Darstellung:

Listing 4.1: Beispielhafte Darstellung einer Netzwerkmetrik

```
# HELP network_device_up Current availability status
# TYPE network_device_up gauge
network_device_up{name="Router",host="192.0.2.1",port="443"} 1.0
network_device_latency_ms{name="Router",host="192.0.2.1",port
="443"} 12.4
```

Das Format ermöglicht die einfache Integration mit Prometheus-kompatiblen Monitoring-Systemen (Anforderung F4).

4.5 Konzeption der Zeitreihenspeicherung

Der Zeitreihenspeicher übernimmt die Persistierung der Metriken als Zeitreihen. Jede Metrik wird mit einem Zeitstempel versehen und in einer optimierten Datenbankstruktur gespeichert.

4.5.1 Speichermodell

Das Speichermodell umfasst folgende Entitäten:

- **Metrik:** Name, Typ, Labels
- **Messwert:** Zeitstempel, numerischer Wert
- **Prüfergebnis:** Ziel, Status, Latenz, Prüfzeitpunkt, Fehlermeldung
- **Incident:** Alert-Informationen, Status, Zeitstempel

Die Speicherung als Zeitreihen ermöglicht die Auswertung aktueller und historischer Werte (Anforderung F5). Das Modell orientiert sich an etablierten Zeitreihen-Datenmodellen [15].

4.5.2 Retentionspolitik

Die Retentionspolitik definiert, wie lange Messwerte und Incidents gespeichert werden. Folgende Parameter sind konfigurierbar:

- **Metrik-Retention:** Speicherdauer für Zeitreihendaten (z.B. 30 Tage)
- **Incident-Retention:** Speicherdauer für Incidents (z.B. 90 Tage)

Die Retentionspolitik ermöglicht einen Kompromiss zwischen Speichernutzung und Nachvollziehbarkeit (Anforderung N5, N6).

4.6 Konzeption der Visualisierung

Die Visualisierungskomponente stellt die gespeicherten Metriken und Incidents in Dashboards dar.

4.6.1 Dashboard-Konzeption

Das Dashboard umfasst folgende Bereiche:

- **Übersicht:** Gesamtanzahl der Geräte, Online-Geräte, Offline-Geräte, aktive Incidents
- **Geräteliste:** Tabelle mit allen konfigurierten Zielen und deren aktuellem Status
- **Status-Panels:** Visuelle Darstellung des Erreichbarkeitsstatus (z.B. Ampelsystem)
- **Latenz-Zeitreihen:** Diagramme der TCP-Latenz über die Zeit
- **Incident-Historie:** Tabelle vergangener Störungen mit Dauer und Beschreibung
- **Anwendungsmetriken:** JVM-Metriken, Thread-Anzahl, Speichernutzung

Die Visualisierung ermöglicht einen schnellen operativen Überblick und die Analyse historischer Entwicklungen (Anforderung F6, N5). Die Dashboard-Gestaltung folgt etablierten Prinzipien der Informationsvisualisierung [29].

4.6.2 Alert-Visualisierung

Aktive Alerts werden in einem separaten Bereich dargestellt. Jeder Alert umfasst:

- Alert-Name und Schweregrad
- Betroffenes Gerät
- Startzeitpunkt
- Beschreibung und Zusammenfassung

Die Darstellung erfolgt sortiert nach Schweregrad und Startzeitpunkt, um kritische Zustände priorisiert anzuzeigen.

4.7 Konzeption der Alert-Komponente

Die Alert-Komponente erkennt kritische Zustände und leitet Benachrichtigungen weiter.

4.7.1 Alert-Regeln

Alert-Regeln definieren Bedingungen, bei deren Erfüllung ein Alert ausgelöst wird. Folgende Regeltypen sind vorgesehen:

- **Gerät nicht erreichbar:** Status-Metrik ist 0 für einen definierten Zeitraum (z.B. 30 Sekunden)
- **Latenz zu hoch:** Latenz-Metrik überschreitet einen definierten Grenzwert (z.B. 200 ms) für einen definierten Zeitraum
- **Monitoring-Dienst nicht verfügbar:** Metrik-Endpunkt ist nicht erreichbar

Die Regeln werden kontinuierlich auf den gespeicherten Zeitreihen ausgewertet (Anforderung F7). Die Regeldefinition orientiert sich an etablierten Alerting-Konzepten [28].

4.7.2 Alert-Weiterleitung

Bei Auslösung eines Alerts wird eine Benachrichtigung an den Incident-Speicher weitergeleitet. Die Weiterleitung umfasst:

- Alert-Informationen (Name, Schweregrad, Labels, Annotations)
- Zeitstempel (Startzeitpunkt, optional Auflösungszeitpunkt)
- Status (Firing oder Resolved)

Die Alert-Weiterleitung erfolgt über einen Webhook-Endpunkt der REST-Schnittstelle.

4.8 Konzeption des Incident-Speichers

Der Incident-Speicher übernimmt die persistente Speicherung von Alerts als Incidents.

4.8.1 Incident-Modell

Jedes Incident umfasst folgende Attribute:

- **Identifikation:** Eindeutige ID und Alert-Fingerprint
- **Alert-Informationen:** Name, Schweregrad, Beschreibung
- **Betroffenes Gerät:** Name, Host, Port
- **Status:** Firing oder Resolved
- **Zeitstempel:** Startzeitpunkt und optional Auflösungszeitpunkt
- **Zusammenfassung und Beschreibung:** Kurz- und Langbeschreibung des Incidents

Das Incident-Modell ermöglicht die langfristige Nachverfolgung von Störungen (Anforderung F8). Das Modell orientiert sich an ITIL-basierten Incident-Management-Konzepten [1].

4.8.2 Incident-Lifecycle

Der Lifecycle eines Incidents beschreibt die Verarbeitung eines durch das Monitoring-System erkannten kritischen Zustands. Ausgangspunkt ist ein von Alertmanager weitergeleiteter Alert mit dem Status `firing`. Um doppelte Einträge zu vermeiden, wird zunächst der Fingerprint des eingehenden Alerts geprüft. Der Fingerprint dient dabei zur eindeutigen Zuordnung wiederholter Benachrichtigungen zu demselben Ereignis.

Existiert zu diesem Fingerprint noch kein aktives Incident, wird ein neuer Incident-Datensatz angelegt und persistent gespeichert. Bereits vorhandene aktive Incidents mit identischem Fingerprint werden bei weiteren `firing`-Benachrichtigungen nicht erneut erstellt. Stattdessen bleibt das vorhandene Incident im Status `FIRING`.

Der Lifecycle umfasst damit die folgenden Zustände und Verarbeitungsschritte:

1. **Empfang eines Firing-Alerts:** Die Anwendung empfängt einen Alert mit dem Status `firing`.
2. **Fingerprint-Prüfung:** Es wird geprüft, ob bereits ein aktives Incident mit demselben Fingerprint existiert.

4 Konzeption

3. **Erstellung und Speicherung:** Falls kein aktives Incident vorhanden ist, wird ein neuer Incident-Datensatz mit Informationen zum Alert, zum betroffenen Gerät, zum Schweregrad und zum Startzeitpunkt erstellt und in der Datenbank gespeichert.
4. **Aktiver Zustand:** Das Incident befindet sich im Status FIRING, solange der zugehörige Alert aktiv ist. Wiederholte firing-Alerts mit gleichem Fingerprint führen nicht zur Erstellung weiterer Incident-Datensätze.
5. **Auflösung:** Beim Empfang eines Alerts mit dem Status resolved wird das zugehörige aktive Incident auf den Status RESOLVED gesetzt. Zusätzlich wird der Auflösungszeitpunkt gespeichert.

Abbildung 4.3 zeigt den konzipierten Lifecycle eines Incidents. Die optionale Archivierung oder Löschung aufgelöster Incidents nach Ablauf einer Aufbewahrungsfrist ist als mögliche Erweiterung vorgesehen und nicht Bestandteil des dargestellten Kernablaufs.

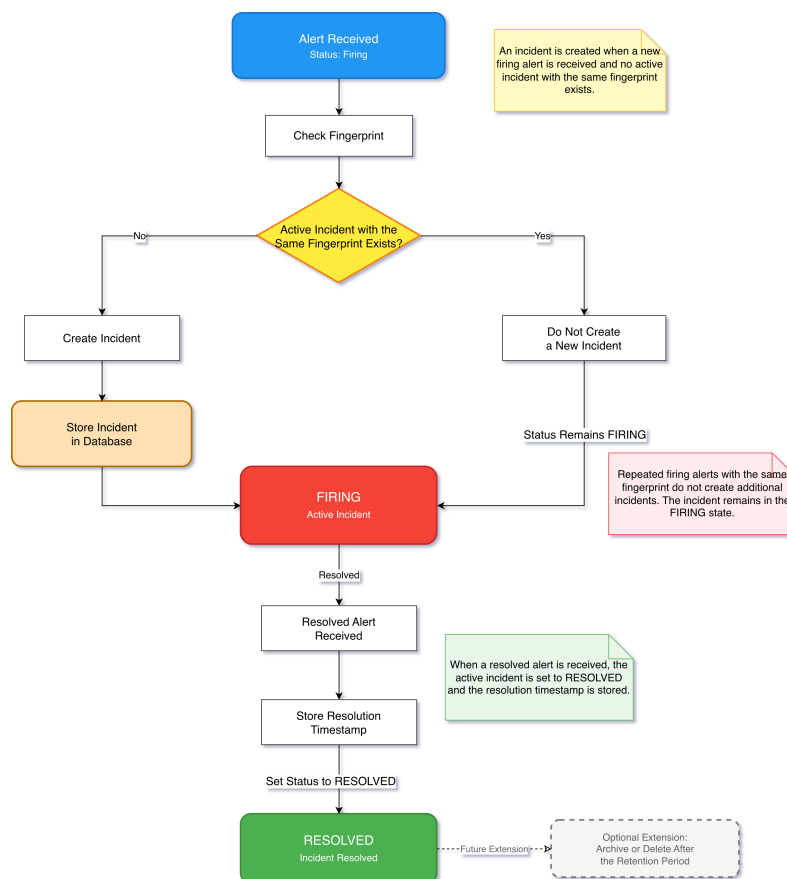


Abbildung 4.3: Lifecycle eines Incidents

4.9 Konzeption der REST-Schnittstelle

Die REST-Schnittstelle ermöglicht die operative Abfrage von Konfiguration, Prüfergebnissen und Incidents.

4.9.1 Endpunkte

Die REST-Schnittstelle umfasst folgende Endpunkte:

- GET /api/status: Dienststatus
- GET /api/devices: Liste der konfigurierten Geräte
- GET /api/devices/{name}: Einzelnes Gerät nach Name
- GET /api/checks/latest: Aktuelle Prüfergebnisse
- GET /api/checks/history: Historische Prüfergebnisse mit Filteroptionen
- POST /api/checks/run: Manuelle Auslösung eines Prüfzyklus
- GET /api/incidents: Alle Incidents
- GET /api/incidents/active: Aktive Incidents
- GET /api/incidents/{id}: Einzelnes Incident nach ID
- POST /api/alerts: Empfang von Alert-Webhooks

Die Endpunkte ermöglichen die Integration mit externen Systemen und die automatisierte Abfrage von Monitoring-Daten (Anforderung F9, F10). Die API-Design-Entscheidungen orientieren sich an den Prinzipien des REST-Architekturstils [8].

4.9.2 Sicherheit

Administrative Endpunkte (z.B. POST /api/checks/run, POST /api/alerts) werden durch Authentifizierung und Autorisierung geschützt. Folgende Sicherheitsmaßnahmen sind vorgesehen:

- **HTTP Basic Authentication:** Benutzername und Passwort für administrative Zugriffe
- **Rollenbasierte Autorisierung:** Unterscheidung zwischen öffentlichen und administrativen Endpunkten
- **Externe Konfiguration der Credentials:** Speicherung der Zugangsdaten in Umgebungsvariablen, nicht im Quellcode

Diese Maßnahmen gewährleisten die Sicherheit der administrativen Endpunkte (Anforderung N7). Die Sicherheitskonzeption folgt etablierten Best Practices für API-Sicherheit [19].

4.10 Konzeption der containerisierten Bereitstellung

Die gesamte Monitoring-Umgebung wird als containerisierte Anwendung bereitgestellt.

4.10.1 Komponenten und Services

Die containerisierte Umgebung umfasst folgende Services:

- **Monitoring-Dienst:** Spring-Boot-Anwendung mit Prüflogik und REST-API
- **Zeitreihenspeicher:** Prometheus-Container für Metrik-Speicherung
- **Visualisierung:** Grafana-Container für Dashboards

4 Konzeption

- **Alert-Komponente:** Alertmanager-Container für Alert-Verarbeitung
- **Demo-Service:** Optionaler NGINX-Container für Testzwecke

Alle Services werden über ein gemeinsames Docker-Netzwerk verbunden. Die Containerisierung folgt etablierten Best Practices für Docker-Anwendungen [12].

4.10.2 Netzwerk und Volumes

Die Konzeption umfasst:

- **Docker-Netzwerk:** Ermöglicht die Kommunikation der Services über ihre Servicenamen
- **Persistente Volumes:** Speicherung von Zeitreihendaten, Incident-Daten und Dashboard-Konfigurationen
- **Konfigurations-Volumes:** Einbinden von Konfigurationsdateien in die Container

Diese Konzeption ermöglicht eine reproduzierbare Bereitstellung (Anforderung N3). Die Volume-Konzeption orientiert sich an etablierten Patterns für persistente Daten in Containern [13].

4.11 Zusammenfassung

Die Konzeption beschreibt eine modulare, technologieunabhängige Architektur für ein leichtgewichtiges Netzwerk-Monitoring-System. Die entwickelten Lösungsansätze schließen die in Kapitel 3 identifizierten Lücken bestehender Monitoring-Lösungen und erfüllen die ermittelten Anforderungen.

Die zentralen Konzepte umfassen:

- **Modulare Architektur:** Trennung der Verantwortlichkeiten auf mehrere Komponenten
- **Incident-Historie:** Persistente Speicherung von Alerts zur langfristigen Nachverfolgung
- **REST-Schnittstelle:** Umfassende API für operative Abfragen
- **Containerisierung:** Reproduzierbare Bereitstellung mit Docker Compose
- **Leichtgewichtigkeit:** Fokus auf Kernfunktionen ohne Überdimensionierung
- **Robustheit:** Fehlerbehandlung und unabhängige Prüfung einzelner Ziele

Die Konzeption bildet die Grundlage für die Implementierung in Kapitel 5. Die beschriebenen Konzepte sind technologieunabhängig formuliert, können jedoch direkt mit konkreten Technologien wie Java, Spring Boot, Prometheus, Grafana und Docker umgesetzt werden.

5 Implementierung

Dieses Kapitel beschreibt die Implementierung des konzipierten Netzwerk-Monitoring-Systems. Ziel der Implementierung ist es, die in Kapitel 4 beschriebenen Lösungsansätze als funktionsfähigen Proof-of-Concept umzusetzen. Dabei werden die wesentlichen Umsetzungstechnologien, die Struktur der Anwendung, die periodische Prüfung von Netzwerkzielen, die Bereitstellung von Metriken, die REST-Schnittstellen, das Incident-Management sowie die containerisierte Ausführung erläutert.

Es werden ausschließlich ausgewählte Implementierungsdetails dargestellt. Eine vollständige Wiedergabe des Quellcodes ist nicht Ziel dieses Kapitels. Stattdessen werden diejenigen Komponenten und Abläufe betrachtet, die für die technische Umsetzung der Anforderungen und die Nachvollziehbarkeit der Lösung wesentlich sind.

5.1 Umsetzungstechnologien

Für die Umsetzung wurde eine Java-basierte Anwendung gewählt. Das Projekt ist als Maven-Projekt aufgebaut und verwendet eine modulare Paketstruktur. Die aktuelle Implementierung enthält unter anderem die Pakete `config`, `controller`, `model`, `service`, `scheduler`, `alert` und `incident`. Dadurch werden technische Aufgaben und fachliche Verantwortlichkeiten voneinander getrennt [30].

5.1.1 Java und Spring Boot

Java wurde als Programmiersprache gewählt, da die Sprache eine ausgereifte Unterstützung für Netzwerkkommunikation, Nebenläufigkeit, Datenbankzugriffe und serverseitige Anwendungen bietet. Zusätzlich erleichtert die statische Typisierung die Wartbarkeit und Nachvollziehbarkeit der Implementierung.

Spring Boot wurde als Anwendungsrahmen eingesetzt. Die Wahl begründet sich insbesondere durch die vorhandene Unterstützung für:

- REST-basierte HTTP-Schnittstellen,
- externe Anwendungskonfiguration,
- Dependency Injection,
- geplante beziehungsweise periodische Aufgaben,
- Sicherheitskonfiguration,
- Datenbankzugriffe und Persistenz,
- Integration von Monitoring- und Metrikfunktionen.

Die Anwendung wird über eine zentrale Startklasse gestartet. Die fachliche Logik ist dabei nicht in der Startklasse konzentriert, sondern auf mehrere Komponenten verteilt.

5 Implementierung

5.1.2 Maven

Maven übernimmt die Verwaltung der Abhängigkeiten, den Build-Prozess und die Erstellung des ausführbaren Artefakts. Die verwendeten Bibliotheken und Plugins werden in der Datei `pom.xml` definiert.

Durch die deklarative Abhängigkeitsverwaltung kann der Build-Prozess reproduzierbar ausgeführt werden. Dies ist insbesondere für die spätere containerisierte Bereitstellung relevant, da das Docker-Image auf einem definierten Build-Ergebnis basiert.

5.1.3 Persistenz

Für die Persistierung der Prüfergebnisse und Incidents wird eine relationale Datenbank verwendet. Die Datenbank speichert unter anderem:

- konfigurierte Netzwerkziele,
- durchgeführte Prüfungen,
- Status- und Latenzwerte,
- Alert-Informationen,
- den Status eines Incidents,
- Erstellungs- und Auflösungszeitpunkte.

Die Entscheidung für eine relationale Persistenzlösung wurde getroffen, weil Prüfergebnisse und Incidents klar strukturierte Attribute besitzen und über Zeitpunkte, Statuswerte sowie Zielsysteme abgefragt werden können.

5.1.4 Docker und Docker Compose

Die Anwendung wird in einem Docker-Container ausgeführt. Die Datei `Dockerfile` beschreibt den Aufbau des Anwendungsimages. Die weiteren Monitoring-Komponenten werden über eine Docker-Compose-Konfiguration gestartet.

Die containerisierte Bereitstellung bietet folgende Vorteile:

- reproduzierbare Umgebung,
- einheitlicher Start der beteiligten Komponenten,
- Trennung der Anwendung von der lokalen Host-Umgebung,
- einfache Weitergabe des Proof-of-Concept,
- zentrale Definition von Netzwerken, Ports und Volumes.

Die Wahl von Docker Compose ist insbesondere für die lokale Entwicklung, Demonstration und Evaluation geeignet, da mehrere voneinander abhängige Services gemeinsam verwaltet werden können [6].

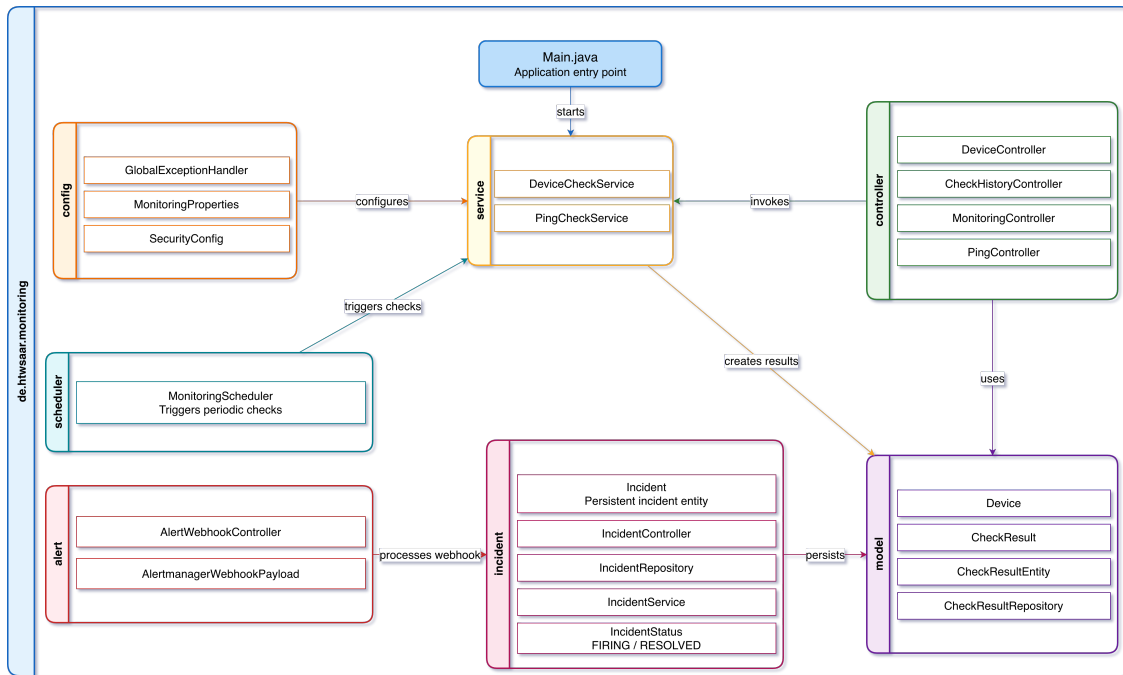


Abbildung 5.1: Struktur der implementierten Anwendung

5.2 Anwendungsstruktur

Die Anwendung ist nach fachlichen und technischen Verantwortlichkeiten strukturiert. Abbildung 5.1 stellt die wesentlichen Module der Implementierung dar.

Die wichtigsten Pakete und ihre Aufgaben sind:

config: Zentrale Konfiguration, Sicherheit und globale Fehlerbehandlung.

controller: REST-Schnittstellen für Geräte, Prüfhistorie, Monitoring und manuelle Prüfungen.

model: Domänenobjekte und Persistenzmodelle für Geräte und Prüfergebnisse.

service: Fachliche Logik für Geräteprüfungen und Erreichbarkeitsmessungen.

scheduler: Zeitgesteuerte Ausführung der regelmäßigen Prüfzyklen.

alert: Verarbeitung von Alert-Webhooks und Abbildung eingehender Alert-Daten.

incident: Verwaltung, Speicherung und Abfrage von Incidents.

Die Struktur unterstützt das Prinzip der Trennung von Verantwortlichkeiten. Controller nehmen Anfragen entgegen, Services führen fachliche Verarbeitung durch und Repository-Komponenten übernehmen den Datenzugriff.

5.3 Konfiguration der Überwachungsziele

Die Überwachungsparameter werden nicht fest in der Prüflogik programmiert, sondern über die Anwendungskonfiguration bereitgestellt. Die Klasse `MonitoringProperties` bündelt die relevanten Konfigurationswerte.

Zu den konfigurierbaren Informationen gehören insbesondere:

5 Implementierung

- Name eines Überwachungsziels,
- Hostname oder IP-Adresse,
- zu prüfender Port,
- Aktivierungsstatus,
- Prüfintervall,
- Verbindungs- beziehungsweise Prüf-Timeout.

Ein vereinfachtes Beispiel der Konfiguration ist in Listing 5.1 dargestellt.

Listing 5.1: Beispielhafte Konfiguration eines Überwachungsziels

```
monitoring:
  interval: 10000
  timeout: 2000
  devices:
    - name: router
      host: 192.0.2.1
      port: 443
      enabled: true
```

Die konkreten Schlüssel und Werte müssen an die tatsächlich verwendete `application.yml` angepasst werden. Entscheidend ist die technische Trennung zwischen Konfiguration und Prüflogik. Dadurch können Überwachungsziele verändert werden, ohne die Java-Klassen neu implementieren zu müssen.

Zusätzliche umgebungsabhängige Werte werden über Umgebungsvariablen beziehungsweise eine Beispieldatei für Umgebungsvariablen bereitgestellt. Sensible Werte werden dadurch nicht zwingend direkt in den Quellcode aufgenommen.

5.4 Implementierung der Geräteprüfung

Die zentrale fachliche Funktion ist die Prüfung eines konfigurierten Netzwerkziels. Die Implementierung ist in den Service-Komponenten der Anwendung gekapselt.

5.4.1 Prüfung der Erreichbarkeit

Für jedes aktivierte Gerät wird eine Verbindung zu der konfigurierten Kombination aus Host und Port aufgebaut. Vor Beginn der Prüfung wird ein Zeitstempel erfasst. Nach Abschluss der Verbindung wird aus der Differenz zwischen Start- und Endzeitpunkt die Dauer des Verbindungsversuchs berechnet.

Der grundsätzliche Ablauf ist:

1. Laden der aktivierten Überwachungsziele.
2. Erfassung des Startzeitpunkts.
3. Aufbau einer TCP-Verbindung zu Host und Port.
4. Anwendung des konfigurierten Timeouts.
5. Ermittlung des Erreichbarkeitsstatus.

6. Berechnung der Latenz bei erfolgreicher Verbindung.
7. Erfassung einer Fehlermeldung bei einem fehlgeschlagenen Versuch.
8. Speicherung und Veröffentlichung des Prüfergebnisses.

Ein vereinfachter Ausschnitt des Ablaufs ist in Listing 5.2 dargestellt.

Listing 5.2: Vereinfachter Ablauf einer Geräteprüfung

```
long start = System.nanoTime();

try {
    // Verbindung zu Host und Port mit Timeout herstellen
    boolean reachable = checkConnection(device);

    long duration = System.nanoTime() - start;
    long latencyMs = duration / 1_000_000;

    saveResult(device, reachable, latencyMs, null);
} catch (Exception exception) {
    long duration = System.nanoTime() - start;
    long latencyMs = duration / 1_000_000;

    saveResult(device, false, latencyMs, exception.getMessage());
}
```

Der dargestellte Code ist bewusst verkürzt und dient ausschließlich der Erläuterung des Prinzips. Die konkrete Implementierung enthält zusätzliche Verarbeitungsschritte für Persistenz, Metrikaktualisierung und Fehlerbehandlung.

5.4.2 Fehlerbehandlung

Ein einzelnes nicht erreichbares Ziel darf den Prüfzyklus der übrigen Ziele nicht beenden. Deshalb wird die Prüfung pro Ziel isoliert behandelt. Netzwerkfehler und Timeouts werden als fehlgeschlagene Prüfergebnisse erfasst.

Mögliche Fehlerfälle sind:

- Verbindungs-Timeout,
- verweigerte Verbindung,
- nicht auflösbarer Hostname,
- ungültige Konfiguration,
- unerwartete Ausnahme während des Verbindungsaufbaus.

Die Klasse `GlobalExceptionHandler` übernimmt zusätzlich die zentrale Behandlung von Fehlern, die während der Verarbeitung von HTTP-Anfragen auftreten. Dadurch können Fehlerantworten einheitlich an API-Clients zurückgegeben werden.

Abbildung 5.2: Implementierter Ablauf eines periodischen Prüfzyklus

5.5 Zeitgesteuerte Ausführung

Die regelmäßige Ausführung der Geräteprüfungen wird durch die Klasse `MonitoringScheduler` realisiert. Der Scheduler startet in dem konfigurierten Intervall einen Prüfzyklus.

Abbildung 5.2 zeigt den vereinfachten Ablauf.

Die Ablaufkette besteht aus:

1. Scheduler erhält ein zeitgesteuertes Ausführungssignal.
2. Der Prüfservice lädt die konfigurierten Ziele.
3. Nicht aktivierte Ziele werden übersprungen.
4. Aktivierte Ziele werden einzeln geprüft.
5. Die Ergebnisse werden gespeichert und als aktuelle Zustände bereitgestellt.

Zusätzlich existiert eine Möglichkeit, einen Prüfzyklus über einen REST-Endpunkt manuell auszulösen. Diese Funktion ist für Tests, Demonstrationen und die Fehlersuche relevant.

5.6 Modellierung und Persistierung

Die Modellklassen bilden die fachlichen Objekte des Systems ab. Im aktuellen Projekt existieren unter anderem Klassen für Geräte, Prüfergebnisse und deren persistierte Repräsentationen.

5.6.1 Gerätemodell

Das Gerätemodell enthält die Informationen, die für eine Erreichbarkeitsprüfung erforderlich sind. Dazu gehören mindestens:

- eine eindeutige Identifikation,
- ein Anzeigename,
- Hostname oder IP-Adresse,
- Portnummer,
- Aktivierungsstatus.

Die Trennung zwischen Gerätemodell und Prüfergebnis ermöglicht es, eine Gerätekonfiguration unabhängig von einzelnen Messungen zu verwalten.

5.6.2 Prüfergebnis

Ein Prüfergebnis beschreibt den Zustand eines Geräts zu einem bestimmten Zeitpunkt. Es umfasst:

- das geprüfte Gerät,
- den Erreichbarkeitsstatus,
- die gemessene Latenz,
- den Prüfzeitpunkt,
- eine optionale Fehlermeldung.

Für die langfristige Speicherung wird das Ergebnis in einer persistierbaren Entität abgebildet. Ein Repository ermöglicht den Zugriff auf aktuelle und historische Ergebnisse.

Listing 5.3: Vereinfachtes Modell eines Prüfergebnisses

```
public class CheckResult {
    private String deviceName;
    private boolean reachable;
    private long latencyMs;
    private Instant timestamp;
    private String errorMessage;
}
```

Der Code ist exemplarisch verkürzt. Die konkrete Implementierung kann zusätzliche Konstruktoren, Zugriffsmethoden, Validierungen oder Persistenzannotationen enthalten.

5.7 Implementierung der REST-Schnittstellen

Die REST-Schnittstellen sind in mehreren Controller-Klassen organisiert. Diese Aufteilung verhindert, dass eine einzelne Controller-Klasse sämtliche Funktionen des Systems enthält.

5.7.1 Geräte- und Monitoring-Schnittstellen

Der DeviceController stellt Funktionen für die Abfrage der konfigurierten Geräte bereit. Der MonitoringController übernimmt allgemeine Monitoring-Funktionen, beispielsweise die Abfrage des Dienststatus oder die manuelle Auslösung eines Prüfzyklus.

Der CheckHistoryController stellt historische Prüfergebnisse bereit. Dadurch kann ein Client aktuelle Zustände und vergangene Messwerte getrennt abfragen.

Ein vereinfachtes Beispiel eines REST-Endpunkts ist in Listing 5.4 dargestellt.

Listing 5.4: Vereinfachter REST-Endpunkt zur Abfrage der Prüfhistorie

```
@GetMapping("/history")
public List<CheckResult> getHistory() {
    return checkHistoryService.findAll();
}
```

Die konkrete URL-Struktur und die Rückgabetypen müssen an die tatsächlich implementierten Controller angepasst werden.

5.7.2 Manuelle Prüfungen

Für Test- und Diagnosezwecke kann ein Prüfzyklus manuell über eine HTTP-Anfrage ausgelöst werden. Dadurch kann geprüft werden, ob Änderungen an der Konfiguration oder am Zielsystem unmittelbar erkannt werden.

Die manuelle Auslösung ist insbesondere für den Proof-of-Concept wichtig, da sie eine reproduzierbare Demonstration einzelner Systemzustände ermöglicht.

5.7.3 Sicherheit der API

Die Klasse `SecurityConfig` enthält die Sicherheitskonfiguration der Anwendung. Administrative oder verändernde Endpunkte werden gegenüber rein lesenden Endpunkten unterschieden.

Die Sicherheitskonfiguration verfolgt insbesondere folgende Ziele:

- Schutz administrativer Funktionen,
- Vermeidung unautorisierter manueller Prüfungen,
- zentrale Definition der Zugriffsregeln,
- externe Bereitstellung sensibler Konfigurationswerte.

Die tatsächliche Absicherung der einzelnen Endpunkte sollte in der Dokumentation exakt entsprechend der implementierten Security-Konfiguration beschrieben werden.

5.8 Metriken und Monitoring-Integration

Die Implementierung stellt die Ergebnisse der Geräteprüfungen als Monitoring-Daten bereit. Für jedes Gerät werden mindestens der Erreichbarkeitsstatus und die Latenz berücksichtigt.

Ein vereinfachtes Metrikmodell ist:

- `network_device_up`: Wert 1 für erreichbar und 0 für nicht erreichbar.
- `network_device_latency_ms`: Dauer des letzten Verbindungsversuchs in Millisekunden.

Die Geräte werden über Labels oder vergleichbare Merkmale unterschieden. Dadurch können einzelne Geräte in Abfragen und Dashboards separat dargestellt werden.

Listing 5.5: Beispielhafte Darstellung exportierter Monitoring-Metriken

```
# HELP network_device_up Current availability status
# TYPE network_device_up gauge
network_device_up{device="router"} 1

# HELP network_device_latency_ms Last connection latency
# TYPE network_device_latency_ms gauge
network_device_latency_ms{device="router"} 12
```

Die konkrete Benennung der Metriken und Labels muss mit dem tatsächlichen Export der Anwendung übereinstimmen. Für die technische Integration werden die im Projekt konfigurierten Monitoring-Komponenten verwendet.

5.9 Alert-Webhooks und Incident-Management

Die Alert-Verarbeitung ist in einem eigenen Paket organisiert. Der `AlertWebhookController` empfängt eingehende Alert-Nachrichten. Das Datenmodell `AlertmanagerWebhookPayload` bildet die Struktur des empfangenen Webhook-Inhalts ab.

5.9.1 Verarbeitung eingehender Alerts

Beim Empfang eines Alerts werden die relevanten Informationen aus dem Payload extrahiert. Dazu zählen beispielsweise:

- Alert-Name,
- Status des Alerts,
- Labels,
- Annotations,
- betroffener Überwachungsdienst oder Zielname,
- Startzeitpunkt,
- optionaler Auflösungszeitpunkt.

Anschließend wird der Alert an den `IncidentService` weitergeleitet. Die Zuständigkeit für die Geschäftslogik bleibt somit im Service und wird nicht direkt im Controller implementiert.

5.9.2 Incident-Modell

Das Incident-Modell wird durch die Klasse `Incident` repräsentiert. Der Status wird über das Enum `IncidentStatus` modelliert. Dadurch sind nur definierte Zustände wie beispielsweise `FIRING` und `RESOLVED` zulässig.

Das Repository stellt den Datenzugriff bereit. Der `IncidentService` implementiert die fachliche Verarbeitung, insbesondere:

- Erstellen eines neuen Incidents,
- Erkennen bereits vorhandener aktiver Incidents,
- Aktualisieren eines bestehenden Incidents,
- Verarbeitung eines aufgelösten Alerts,
- Abfrage aktiver und historischer Incidents.

Durch die Verwendung eines Alert-Fingerprints oder einer vergleichbaren Identifikation kann verhindert werden, dass derselbe aktive Alert bei wiederholter Zustellung mehrfach als neues Incident gespeichert wird.

5.9.3 REST-Abfrage der Incidents

Der IncidentController stellt die Incident-Historie für externe Clients bereit. Typische Abfragen sind:

- alle Incidents,
- aktive Incidents,
- ein einzelnes Incident,
- Incidents nach Status oder Zeitraum.

Die Incident-Historie ergänzt die Zeitreihendaten um eine fachlich interpretierbare Darstellung von Störungen.

5.10 Globale Fehlerbehandlung

Die Klasse GlobalExceptionHandler bündelt die Behandlung ausgewählter Fehler, die während der Verarbeitung von REST-Anfragen auftreten. Ohne eine zentrale Fehlerbehandlung könnten unterschiedliche Controller inkonsistente Antwortformate liefern.

Eine einheitliche Fehlerantwort kann beispielsweise folgende Informationen enthalten:

- HTTP-Statuscode,
- Zeitpunkt des Fehlers,
- Fehlernachricht,
- betroffener Request-Pfad.

Die zentrale Behandlung verbessert die Wartbarkeit der API und erleichtert die Diagnose während des Betriebs.

5.11 Containerisierte Bereitstellung

Die Anwendung wird über ein Dockerfile in ein Container-Image überführt. Der Build-Prozess umfasst typischerweise:

1. Bereitstellung einer geeigneten Java-Laufzeitumgebung.
2. Kopieren des erzeugten Anwendungspakets in das Image.
3. Festlegen des Startbefehls.
4. Öffnen beziehungsweise Dokumentieren des Anwendungsports.

Die Datei `docker-compose.yml` beschreibt die gemeinsame Ausführung der Anwendung und der unterstützenden Monitoring-Komponenten.

Ein vereinfachter Ausschnitt ist in Listing 5.6 dargestellt.

Listing 5.6: Vereinfachtes Beispiel der containerisierten Bereitstellung

```
services:
  monitoring:
    build: .
    ports:
      - "8080:8080"

  prometheus:
    image: prom/prometheus

  grafana:
    image: grafana/grafana
```

Der tatsächliche Compose-Ausschnitt muss entsprechend der aktuellen Projektdatei angepasst werden. Insbesondere Namen, Ports, Volumes, Netzwerke und Umgebungsvariablen dürfen in der finalen Arbeit nicht abweichend vom Repository dargestellt werden.

5.11.1 Persistente Daten

Daten, die den Neustart eines Containers überleben müssen, werden über Volumes oder gemountete Verzeichnisse gespeichert. Dies betrifft insbesondere:

- Zeitreihendaten,
- Incident-Daten,
- Grafana-Konfiguration,
- Monitoring-Konfigurationen.

Damit wird verhindert, dass bei jedem Neustart der Container sämtliche historischen Daten verloren gehen.

5.12 Proof-of-Concept

Die Implementierung stellt einen funktionsfähigen Proof-of-Concept dar. Die wesentlichen Kernfunktionen aus der Konzeption sind technisch umgesetzt:

- konfigurierbare Überwachungsziele,
- periodische Netzwerkprüfungen,
- Speicherung von Prüfergebnissen,
- Bereitstellung einer REST-Schnittstelle,
- Bereitstellung von Monitoring-Metriken,
- Empfang von Alert-Webhooks,
- Verwaltung einer Incident-Historie,
- containerisierte Ausführung.

Der Proof-of-Concept erhebt nicht den Anspruch, sämtliche Anforderungen eines vollständigen Enterprise-Monitoring-Systems abzudecken. Funktionen wie hochverfügbare Speicherung, verteilte Skalierung, automatische Service-Discovery oder umfangreiche Benutzer- und Rollenverwaltung sind nicht oder nur teilweise umgesetzt.

Die Implementierung zeigt jedoch, dass die in Kapitel 4 beschriebene Architektur mit den gewählten Technologien technisch realisierbar ist.

5.13 Zusammenfassung

In diesem Kapitel wurde die Umsetzung des konzipierten Netzwerk-Monitoring-Systems beschrieben. Die Java-Anwendung ist modular aufgebaut und trennt Konfiguration, Controller, Domänenmodell, Services, Scheduler, Alert-Verarbeitung und Incident-Management.

Die Geräteprüfung erfolgt zeitgesteuert und ermittelt den Erreichbarkeitsstatus sowie die Latenz eines konfigurierten Netzwerkziels. Die Ergebnisse werden gespeichert, als Monitoring-Metriken bereitgestellt und über REST-Schnittstellen zugänglich gemacht. Eingehende Alerts werden über einen Webhook verarbeitet und als Incidents persistiert.

Durch Docker und Docker Compose kann die Anwendung gemeinsam mit den unterstützenden Monitoring-Komponenten reproduzierbar gestartet werden. Damit ist die technische Umsetzbarkeit der in Kapitel 4 beschriebenen Lösungsansätze nachgewiesen. Die Bewertung der erreichten Funktionalität und der verbleibenden Einschränkungen erfolgt in Kapitel ??.

6 Evaluation

7 Zusammenfassung und Ausblick

Literatur

- [1] AXELOS. *ITIL Foundation: ITIL 4 Edition*. TSO (The Stationery Office), 2019.
- [2] Prometheus Authors. *Alertmanager*. Zugriff am 16.08.2026. 2026. URL: <https://prometheus.io/docs/alerting/latest/alertmanager/>.
- [3] Prometheus Authors. *Exposition Formats*. Zugriff am 16.08.2026. 2026. URL: https://prometheus.io/docs/instrumenting/exposition_formats/.
- [4] Brian Brazil. *Prometheus: Up & Running*. O'Reilly, 2019.
- [5] Brian Brazil. *Prometheus: Up & Running*. O'Reilly Media, 2019.
- [6] Docker Inc. *Docker Compose Documentation*. 2026. URL: <https://docs.docker.com/compose/> (besucht am 15.09.2026).
- [7] Docker Inc. *Docker Compose*. <https://docs.docker.com/compose/> (Zugriff am 16.08.2026). 2026.
- [8] Roy Thomas Fielding. *Architectural Styles and the Design of Network-based Software Architectures*. University of California, Irvine, 2000.
- [9] Grafana Labs. *Configure the Prometheus Data Source*. <https://grafana.com/docs/grafana/latest/datasources/prometheus/configure/> (Zugriff am 16.08.2026). 2026.
- [10] Gregor Hohpe und Bobby Woolf. *Enterprise Integration Patterns*. Addison-Wesley, 2003.
- [11] Jez Humble und David Farley. *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Addison-Wesley, 2010.
- [12] Docker Inc. *Docker Best Practices*. Zugriff am 16.08.2026. 2026. URL: https://docs.docker.com/develop/develop-images/dockerfile_best-practices/.
- [13] Docker Inc. *Use Volumes*. Zugriff am 16.08.2026. 2026. URL: <https://docs.docker.com/storage/volumes/>.
- [14] JUnit Team. *JUnit 5 User Guide*. <https://junit.org/junit5/docs/current/user-guide/> (Zugriff am 16.08.2026). 2026.
- [15] Martin Kleppmann. *Designing Data-Intensive Applications*. O'Reilly Media, 2017.
- [16] Donald E. Knuth. „Computer Programming as an Art“. In: *Communications of the ACM* 17.12 (1974), S. 667–673.
- [17] Grafana Labs. *Configure the Prometheus Data Source*. Zugriff am 16.08.2026. 2026. URL: <https://grafana.com/docs/grafana/latest/datasources/prometheus/configure/>.
- [18] Axel van Lamsweerde. *Requirements Engineering: From System Goals to UML Models to Software Specifications*. Wiley, 2009.
- [19] OWASP. *API Security Best Practices*. Zugriff am 16.08.2026. 2026. URL: <https://owasp.org/www-project-api-security/>.
- [20] Mary Poppendieck und Tom Poppendieck. *Lean Software Development: An Agile Toolkit*. Addison-Wesley, 2003.

- [21] Prometheus Authors. *Alerting Overview*. <https://prometheus.io/docs/alerting/latest/overview/> (Zugriff am 16.08.2026). 2026.
- [22] Prometheus Authors. *Alertmanager*. <https://prometheus.io/docs/alerting/latest/alertmanager/> (Zugriff am 16.08.2026). 2026.
- [23] Prometheus Authors. *Configuration*. <https://prometheus.io/docs/prometheus/latest/configuration/configuration/> (Zugriff am 16.08.2026). 2026.
- [24] Prometheus Authors. *Metric Types*. https://prometheus.io/docs/concepts/metric_types/ (Zugriff am 16.08.2026). 2026.
- [25] Mark Richards und Neal Ford. *Software Architecture Patterns*. O'Reilly Media, 2015.
- [26] Spring Team. *Metrics :: Spring Boot*. <https://docs.spring.io/spring-boot/reference/actuator/metrics.html> (Zugriff am 16.08.2026). 2026.
- [27] Spring Team. *Task Execution and Scheduling*. <https://docs.spring.io/spring-framework/reference/integration/scheduling.html> (Zugriff am 16.08.2026). 2026.
- [28] Google SRE Team. *Alerting Best Practices*. Zugriff am 16.08.2026. 2026. URL: <https://sre.google/sre-book/monitoring-distributed-systems/>.
- [29] Colin Ware. *Information Visualization: Perception for Design*. Morgan Kaufmann, 2012.
- [30] Xudong Zhang. *Netzwerk-Monitoring-Dashboard mit Automatisierung*. GitHub Repository. Interne Projektquelle. 2026.

Abbildungsverzeichnis

4.1	Konzeptionelle Übersicht der Gesamtarchitektur	20
4.2	Ablauf einer Erreichbarkeitsprüfung	23
4.3	Lifecycle eines Incidents	28
5.1	Struktur der implementierten Anwendung	33
5.2	Implementierter Ablauf eines periodischen Prüfzyklus	36

Tabellenverzeichnis

3.1	Vergleichende Anforderungsabdeckung bestehender Monitoring-Lösungen	17
-----	---	----

Listings

4.1	Beispielhafte Darstellung einer Netzwerkmetrik	25
5.1	Beispielhafte Konfiguration eines Überwachungsziels	34
5.2	Vereinfachter Ablauf einer Geräteprüfung	35
5.3	Vereinfachtes Modell eines Prüfergebnisses	37
5.4	Vereinfachter REST-Endpunkt zur Abfrage der Prüfhistorie	37
5.5	Beispielhafte Darstellung exportierter Monitoring-Metriken	38
5.6	Vereinfachtes Beispiel der containerisierten Bereitstellung	41

Abkürzungsverzeichnis

Anhang

A Erster Abschnitt des Anhangs

In den Anhang gehören „Hintergrundinformationen“, also weiterführende Information, ausführliche Listings, Graphen, Diagramme oder Tabellen, die den Haupttext mit detaillierten Informationen ergänzen.

Dies hier ist ein Blindtext zum Testen von Textausgaben. Wer diesen Text liest, ist selbst schuld. Der Text gibt lediglich den Grauwert der Schrift an. Ist das wirklich so? Ist es gleichgültig, ob ich schreibe: „Dies ist ein Blindtext“ oder „Huardest gefburn“? Kjift – mitnichten! Ein Blindtext bietet mir wichtige Informationen. An ihm messe ich die Lesbarkeit einer Schrift, ihre Anmutung, wie harmonisch die Figuren zueinander stehen und prüfe, wie breit oder schmal sie läuft. Ein Blindtext sollte möglichst viele verschiedene Buchstaben enthalten und in der Originalsprache gesetzt sein. Er muss keinen Sinn ergeben, sollte aber lesbar sein. Fremdsprachige Texte wie „Lorem ipsum“ dienen nicht dem eigentlichen Zweck, da sie eine falsche Anmutung vermitteln. Dies hier ist ein Blindtext zum Testen von Textausgaben. Wer diesen Text liest, ist selbst schuld. Der Text gibt lediglich den Grauwert der Schrift an. Ist das wirklich so? Ist es gleichgültig, ob ich schreibe: „Dies ist ein Blindtext“ oder „Huardest gefburn“? Kjift – mitnichten! Ein Blindtext bietet mir wichtige Informationen. An ihm messe ich die Lesbarkeit einer Schrift, ihre Anmutung, wie harmonisch die Figuren zueinander stehen und prüfe, wie breit oder schmal sie läuft. Ein Blindtext sollte möglichst viele verschiedene Buchstaben enthalten und in der Originalsprache gesetzt sein. Er muss keinen Sinn ergeben, sollte aber lesbar sein. Fremdsprachige Texte wie „Lorem ipsum“ dienen nicht dem eigentlichen Zweck, da sie eine falsche Anmutung vermitteln. Dies hier ist ein Blindtext zum Testen von Textausgaben. Wer diesen Text liest, ist selbst schuld. Der Text gibt lediglich den Grauwert der Schrift an. Ist das wirklich so? Ist es gleichgültig, ob ich schreibe: „Dies ist ein Blindtext“ oder „Huardest gefburn“? Kjift – mitnichten! Ein Blindtext bietet mir wichtige Informationen. An ihm messe ich die Lesbarkeit einer Schrift, ihre Anmutung, wie harmonisch die Figuren zueinander stehen und prüfe, wie breit oder schmal sie läuft. Ein Blindtext sollte möglichst viele verschiedene Buchstaben enthalten und in der Originalsprache gesetzt sein. Er muss keinen Sinn ergeben, sollte aber lesbar sein. Fremdsprachige Texte wie „Lorem ipsum“ dienen nicht dem eigentlichen Zweck, da sie eine falsche Anmutung vermitteln.

Kolophon

Dieses Dokument wurde mit der L^AT_EX-Vorlage für Abschlussarbeiten an der htw saar im Bereich Informatik/Mechatronik-Sensortechnik erstellt (Version 2.25, August 2024). Die Vorlage wurde von Yves Hary und André Miede entwickelt (mit freundlicher Unterstützung von Thomas Kretschmer, Helmut G. Folz und Martina Lehser). Daten: (F)10.95 – (B)426.79135pt – (H)688.5567pt